

Arquitecturas de Datos

Especialización en Internet de las Cosas

Docente:

Lacheski, Martín Anibal

Clase 2:

Modelado de Datos

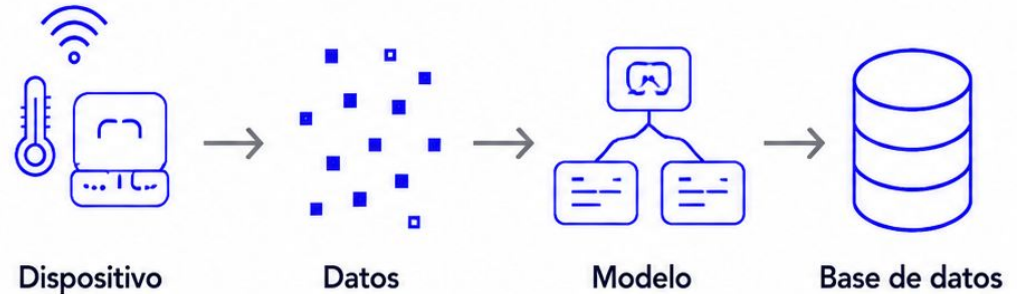


Arquitectura de Datos

De los datos al modelo

En la clase anterior vimos:

- cómo se generan los datos en una solución IoT;
- cómo ingresan al sistema;
- dónde pueden almacenarse;
- cómo persisten;
- qué diferencia existe entre estado actual e histórico.



Hoy vamos a preguntarnos:

¿cómo organizamos correctamente esos datos?



Antes de elegir tablas o tecnologías,
necesitamos comprender qué información representa el sistema.

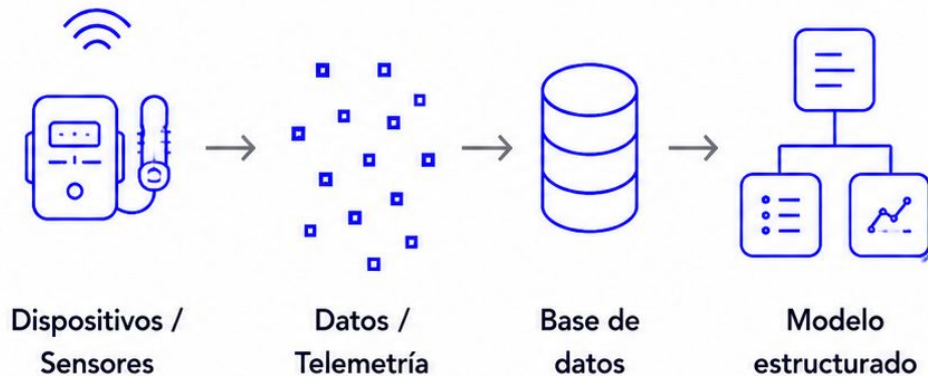
El problema de la clase anterior

Teníamos datos pero todavía no teníamos un modelo

Nuestro dispositivo ya generaba y almacenaba mediciones ambientales junto con la información necesaria para identificarlas y ubicarlas en el tiempo.

Pero todavía quedaban preguntas:

- ¿qué representa cada dato?
- ¿cómo identificamos un dispositivo?
- ¿cómo sabemos dónde está instalado?
- ¿cómo relacionamos un sensor con sus mediciones?
- ¿cómo representamos la información que cambia con el tiempo?



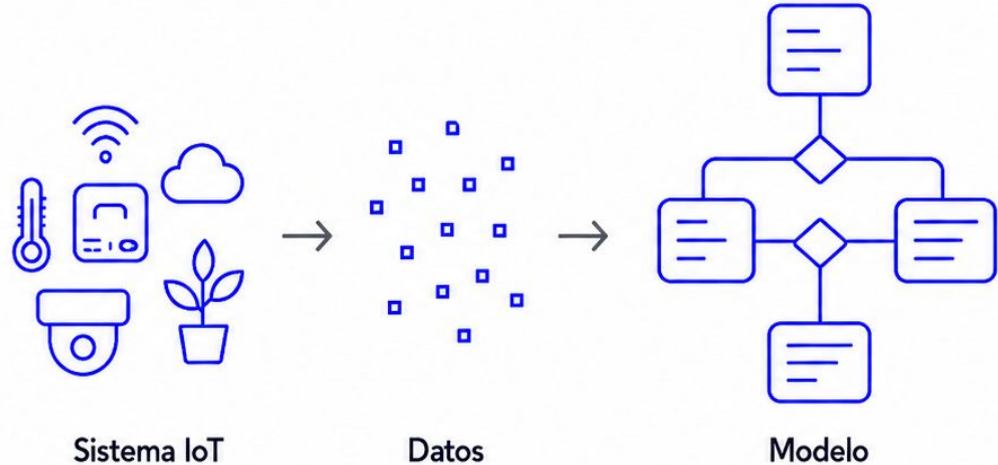
Guardar datos no significa necesariamente tenerlos bien organizados.

Objetivos de la clase

Representar un sistema IoT antes de implementarlo

Vamos a aprender a:

- identificar entidades;
- definir atributos;
- reconocer relaciones;
- establecer cardinalidades;
- identificar restricciones;
- distinguir claves naturales y sustitutas;
- representar estados e historiales;
- transformar requerimientos en un modelo de datos.



Modelar es decidir cómo representar la realidad que necesita comprender nuestro sistema.

Caso guía

Sistema de monitoreo de ambientes

Durante la clase vamos a modelar un sistema que permita administrar:

- organizaciones;
- ubicaciones;
- dispositivos;
- sensores;
- mediciones;
- usuarios;
- configuraciones;
- estados.



Los sensores registrarán variables ambientales a lo largo del tiempo.

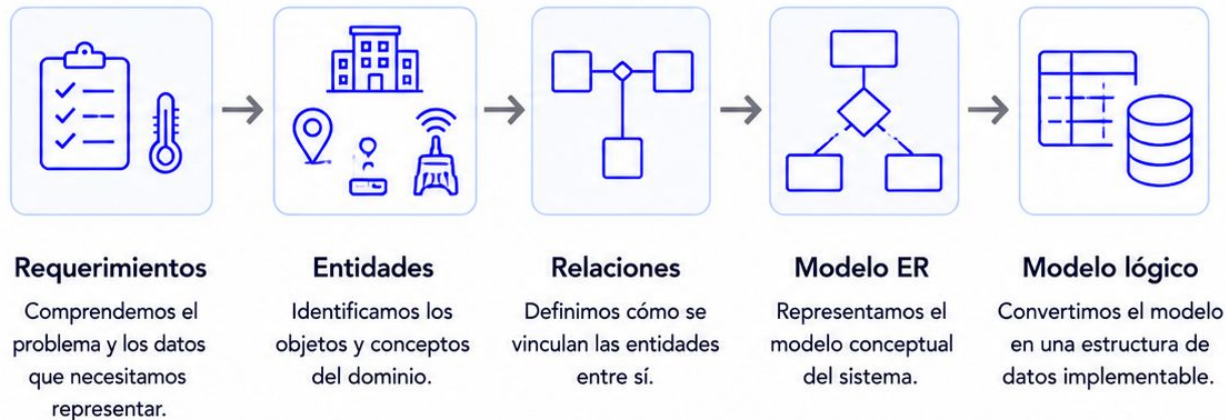


Vamos a construir el modelo progresivamente a partir de los requerimientos.

Arquitectura de Datos

Agenda del día

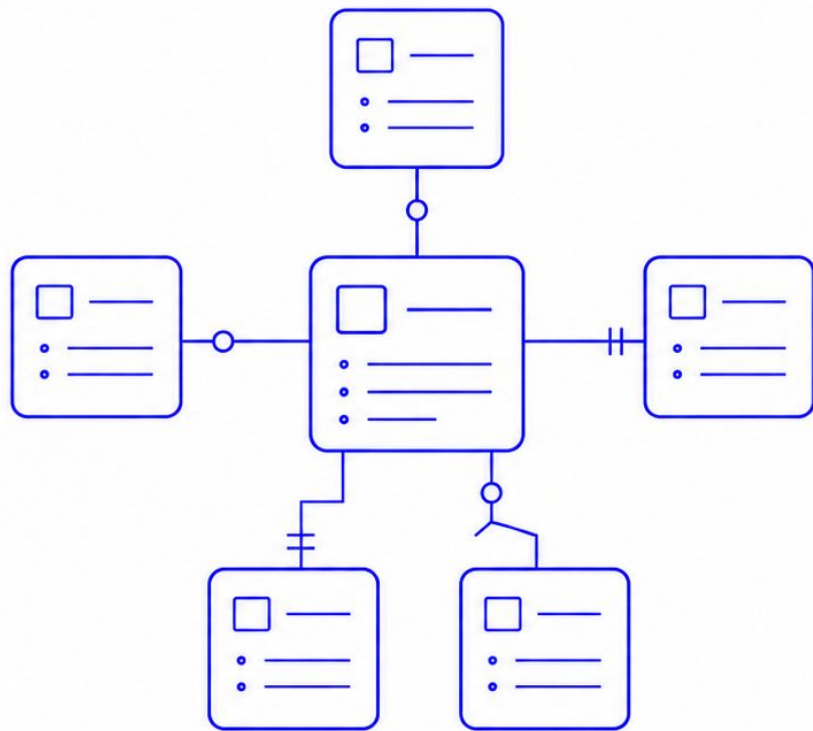
- ¿Qué significa modelar datos?
- Modelado conceptual, lógico y físico.
- Entidades y atributos.
- Identificadores.
- Claves naturales y sustitutas.
- Relaciones y cardinalidades.
- Restricciones.
- Modelo Entidad-Relación.
- Estados e historiales.
- UML aplicado al modelado.
- De los requerimientos al modelo.
- Práctica guiada.



Vamos a recorrer el camino desde un problema del mundo real hasta una estructura de datos capaz de representarlo.

1

Modelado de Datos

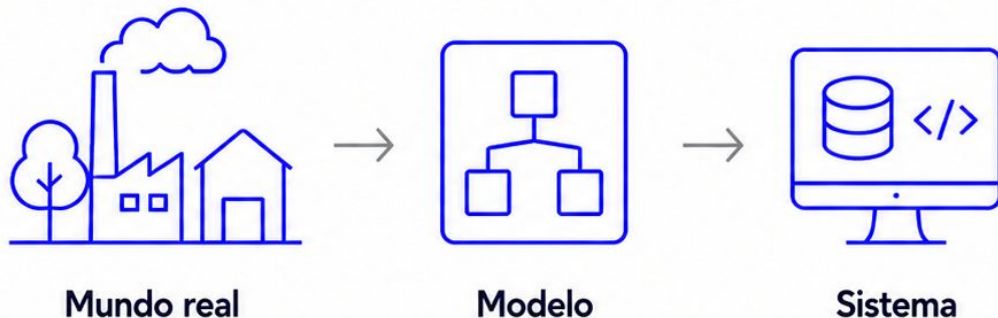


¿Qué es modelar datos?

Representar el problema antes de construir la solución

Modelar datos implica identificar:

- qué información necesita el sistema;
- qué conceptos existen;
- qué características tienen;
- cómo se relacionan;
- qué reglas deben cumplirse.



Un modelo de datos es una representación simplificada de una parte de la realidad.

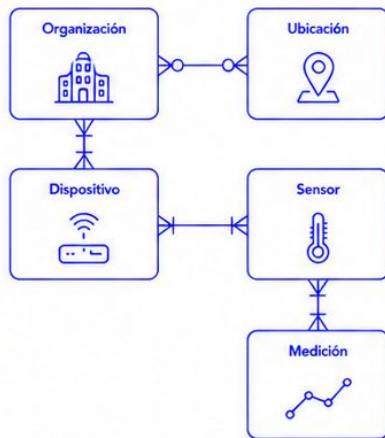
¿Por qué modelar?

Porque los datos no existen aislados

Un buen modelo permite:

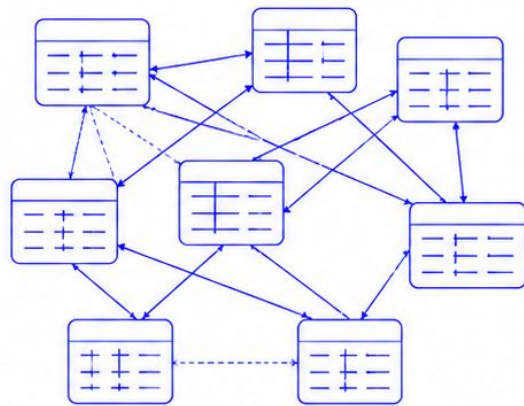
- reducir ambigüedades;
- evitar duplicaciones innecesarias;
- mantener consistencia;
- representar reglas del dominio;
- facilitar las consultas;
- evolucionar el sistema con mayor claridad.

Modelo ordenado



Información clara, relaciones definidas,
consistencia y consultas simples.

Modelo desorganizado



Información confusa, duplicada,
inconsistencias y consultas complejas.



Un problema de modelado suele aparecer
después como un problema de implementación.

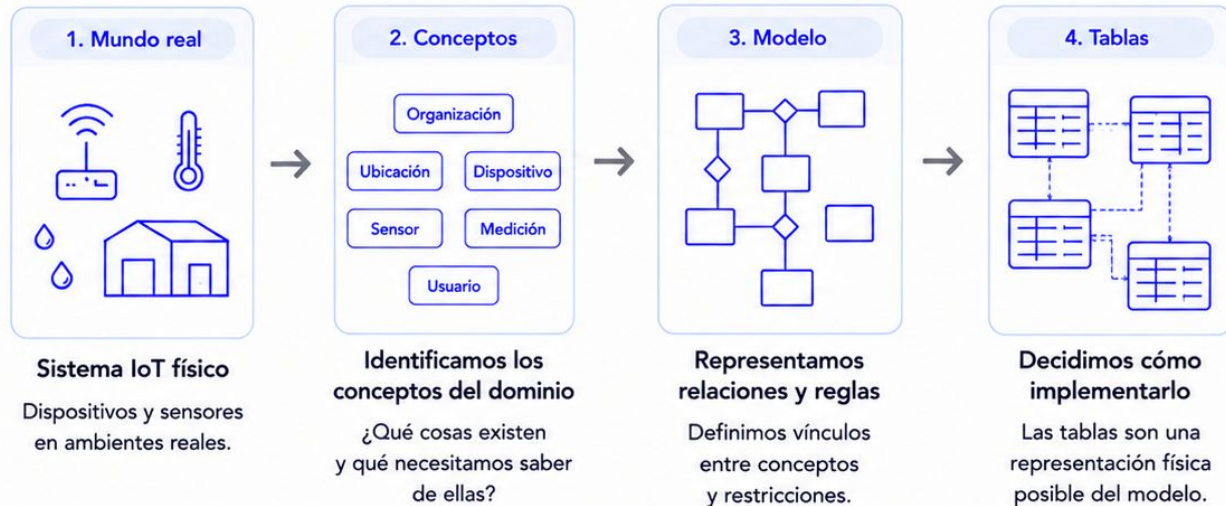
Modelar no es dibujar tablas

Primero necesitamos comprender el dominio

Una tabla es una posible representación física.

Antes debemos preguntarnos:

- ¿qué cosas existen?
- ¿qué necesitamos saber de ellas?
- ¿qué vínculos existen?
- ¿qué reglas se deben respetar?



La primera pregunta no es "¿qué tablas necesito?".
Es "¿qué necesito representar?".

IoT conecta dos mundos

Objetos físicos y representaciones digitales

En el mundo físico tenemos elementos como:

- ambientes;
- dispositivos;
- personas.

En el sistema necesitamos representar digitalmente aquellos elementos que resultan relevantes para la solución.



Dispositivo físico
Mide temperatura y humedad en un ambiente.



Entidad Dispositivo
Representa digitalmente al dispositivo físico y sus atributos.



Datos almacenados

dispositivos				
id	nombre	modelo	ubicacion_id	estado
1	Sensor A1	TH-02	5	activo
...	...	...	...	...

mediciones					
id	dispositivo_id	variable	valor	unidad	timestamp
101	1	temperatura	23.4	°C	2025-08-27 10:15:00
...	...	...	...	...	...

estados				
id	dispositivo_id	estado	desde	hasta
10	1	activo	2025-08-20 09:00:00	NULL
...	...	...	...	...

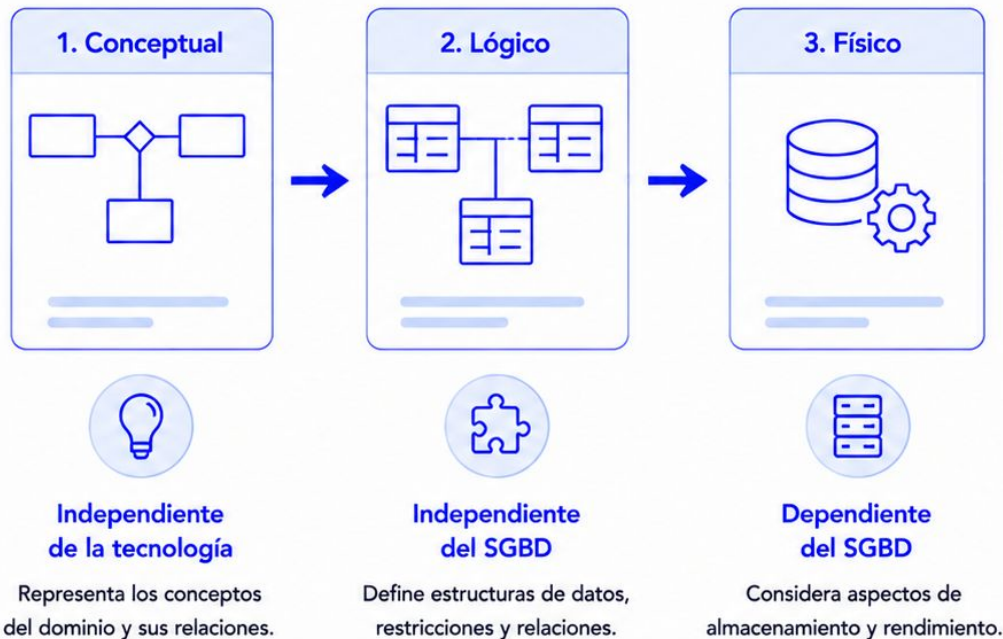
Datos almacenados
Tablas que guardan la información del dispositivo y sus mediciones.



El modelo de datos crea una representación digital del dominio físico.

2

Niveles de Modelado



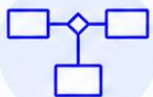
Niveles de modelado

Conceptual, lógico y físico



Conceptual

Describe los conceptos importantes del problema.



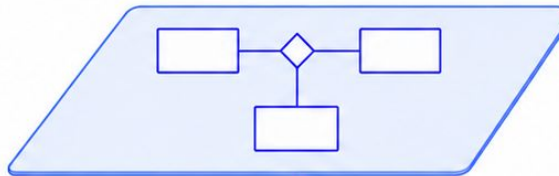
Lógico

Organiza esos conceptos, sus atributos y relaciones.



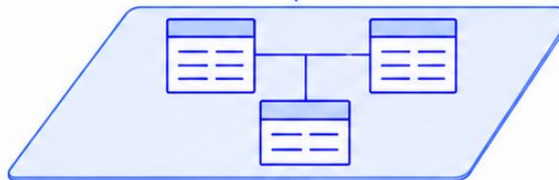
Físico

Define cómo se implementarán en una tecnología concreta.



Conceptual

Qué cosas existen en el dominio.



Lógico

Estructura de datos, atributos y relaciones.



Físico

Implementación en una tecnología concreta.



Vamos de una representación del problema hacia una estructura implementable.

Modelo conceptual

Comprender qué existe

Se enfoca en las entidades principales del caso guía y en las relaciones que existen entre ellas.

Todavía no nos preocupamos por:

- columnas;
- tipos de datos;
- nombres físicos;
- características de PostgreSQL.



El modelo conceptual responde: ¿qué cosas importantes existen y cómo se relacionan?

Modelo lógico

Organizar la información

Incorporamos:

- atributos;
- identificadores;
- relaciones;
- cardinalidades;
- restricciones.



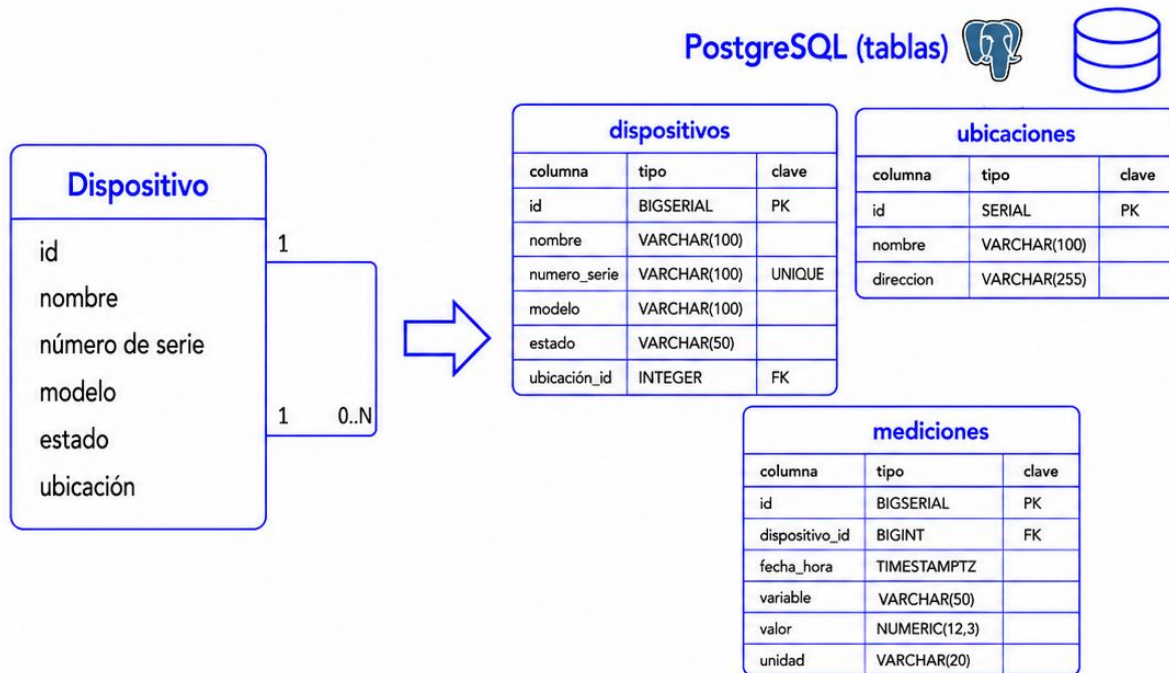
El modelo lógico responde: ¿cómo se estructura la información?

Modelo físico

Preparar la implementación

Define aspectos como:

- tablas;
- columnas;
- tipos de datos;
- claves;
- restricciones;
- nombres concretos;
- características del motor seleccionado.



El modelo físico responde: ¿cómo implementamos el modelo en una tecnología concreta?

Conceptual, lógico y físico

Tres niveles del mismo sistema

1. Conceptual

Dispositivo genera mediciones.



2. Lógico

Dispositivo 1:N Medición



3. Físico

dispositivos
mediciones
con sus respectivas claves y referencias.

dispositivos	
PK	id
	nombre
	numero_serie
	modelo
	estado
	ubicacion_id

mediciones	
PK	id
FK	dispositivo_id
	variable
	valor
	unidad
	timestamp



El problema es el mismo. Lo que cambia es el nivel de detalle.

3

Entidades y Atributos



Organización

Representa a la organización dueña de los dispositivos.



Ubicación

Lugares físicos donde se encuentran los dispositivos.



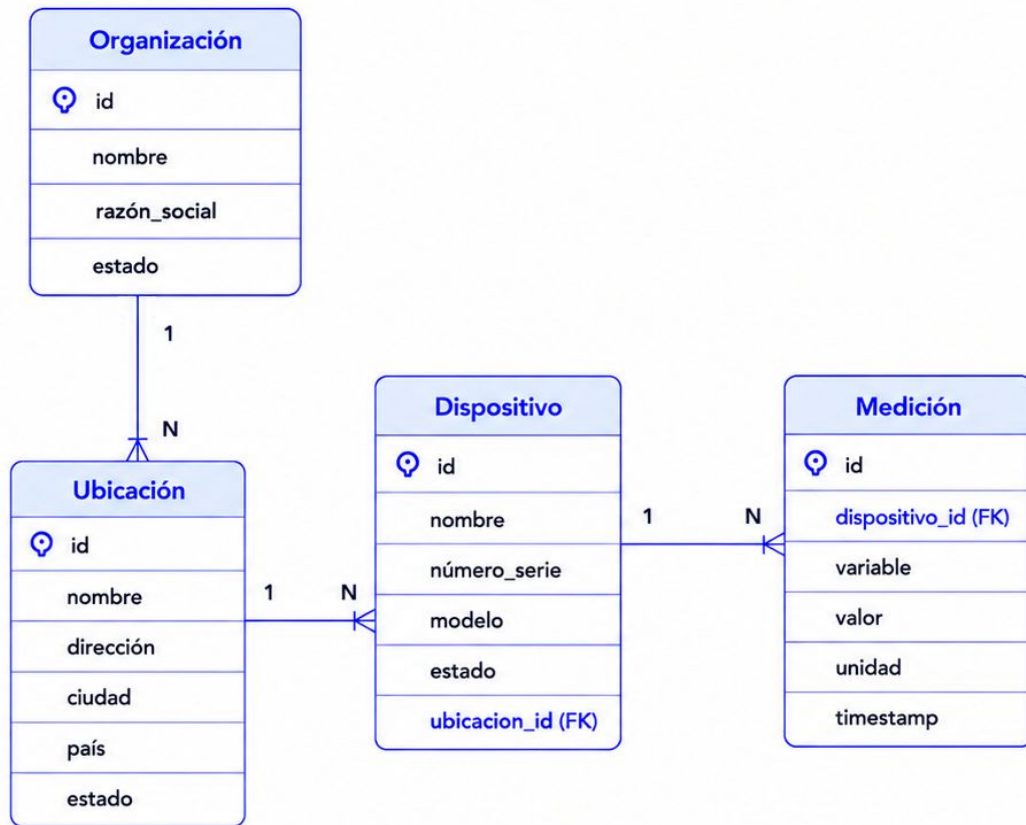
Dispositivo

Equipos que generan mediciones.



Medición

Valores registrados por los dispositivos.



¿Qué es una entidad?

Algo relevante dentro del dominio

Una entidad puede representar:

- un objeto;
- una persona;
- un lugar;
- un concepto;
- un evento.

En nuestro caso podemos reconocer, por ejemplo, **organizaciones, usuarios, ubicaciones, dispositivos y mediciones.**

Organización



Una empresa u organización que posee y gestiona dispositivos.

Usuario



Una persona que accede al sistema y realiza acciones.

Ubicación



Un lugar físico donde se encuentran instalados los dispositivos.

Dispositivo



Un equipo físico que recoge y envía datos.

Medición



Un registro de datos generado por un dispositivo.



Una entidad representa algo sobre lo que necesitamos guardar información.

¿Entidad o atributo?

Una decisión frecuente al modelar

Supongamos que tenemos:

Dispositivo

- nombre;
- número de serie;
- fabricante.

¿Fabricante debería ser simplemente un atributo?

¿O necesitamos una entidad **Fabricante**?

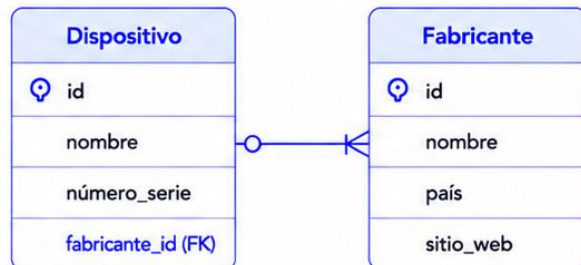
Depende del problema.

Opción 1: Fabricante como atributo



Fabricante se guarda como un atributo dentro de Dispositivo.

Opción 2: Fabricante como entidad



Fabricante es una entidad independiente y se relaciona con Dispositivo.



El modelo debe representar las necesidades reales del sistema, no todas las cosas posibles.

¿Qué es un atributo?

Una característica de una entidad

Un atributo describe algo que necesitamos conocer.

Ejemplo:

Dispositivo

- identificador;
- nombre;
- número de serie;
- modelo;
- fecha de instalación;
- estado.

Dispositivo	
	identificador
	nombre
	número de serie
	modelo
	fecha de instalación
	estado



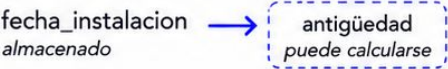
Los atributos describen qué sabemos de cada entidad.

Tipos de atributos

No todos representan lo mismo

Podemos encontrar atributos:

- simples;
- compuestos;
- opcionales;
- obligatorios;
- derivados;
- identificadores.

Dispositivo		
	modelo	atributo simple
	ubicación 	atributo compuesto
	alias	atributo opcional
	numero_serie	atributo obligatorio
	id_dispositivo	atributo identificador
	fecha_instalacion almacenado → 	atributo derivado

fecha_instalacion → almacenado

antigüedad → puede calcularse



No todo dato que podemos calcular necesita almacenarse.

¿Todo debe almacenarse?

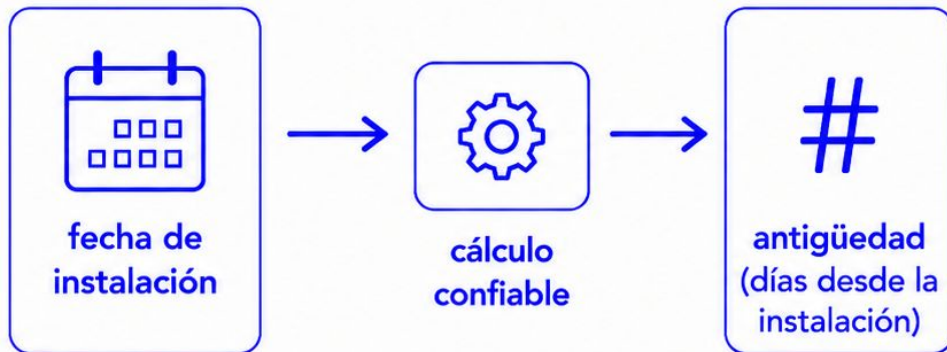
Una decisión de modelado

Supongamos que conocemos:

- fecha de instalación;
- días desde la instalación.

¿Necesitamos guardar ambos?

Si un valor puede obtenerse de manera confiable a partir de otro, almacenarlo también puede generar inconsistencias.



Guardar datos redundantes implica mantenerlos consistentes.

4

Identificadores



Dispositivo	
id	1
nombre	Sensor A1
número_serie	SN-A1-1001
modelo	TempH-200
fecha_instalacion	2024-01-10
estado	activo



Dispositivo	
id	2
nombre	Sensor A2
número_serie	SN-A2-1002
modelo	TempH-200
fecha_instalacion	2024-01-12
estado	activo



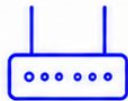
Dispositivo	
id	3
nombre	Sensor B1
número_serie	SN-B1-1003
modelo	TempH-200
fecha_instalacion	2024-01-15
estado	activo

Instancias similares, cada una con su identificador único.

Identificadores

¿Cómo distinguimos una entidad de otra?

Podemos tener cientos o miles de dispositivos y millones de mediciones.



Dispositivos

dispositivo_id	nombre	número_serie
1	Sensor A1	SN-A1-1001
2	Sensor A2	SN-A2-1002
3	Sensor B1	SN-B1-1003
4	Sensor C1	SN-C1-1004
5	Sensor D1	SN-D1-1005
...	...	...



Mediciones

medicion_id	dispositivo_id	variable	valor	timestamp
1001	1	temperatura	22.4	2025-05-01 10:00:00
1002	1	humedad	45.2	2025-05-01 10:01:00
1003	2	temperatura	23.1	2025-05-01 10:00:00
1004	3	temperatura	21.0	2025-05-01 10:00:00
1005	3	humedad	50.8	2025-05-01 10:01:00
...	...	...	...	...

Cada registro necesita poder identificarse de manera inequívoca.

Ejemplos:

- dispositivo_id;
- medicion_id;
- usuario_id;
- organizacion_id.



Sin identificadores claros, las relaciones se vuelven ambiguas.

Clave natural

Un identificador que ya existe en el dominio

Algunos datos pueden identificar naturalmente una entidad.

Ejemplos:

- número de serie;
- dirección MAC;
- código de activo;
- email de usuario.



Una clave natural tiene significado fuera de la base de datos.

¿Siempre conviene usarlas?

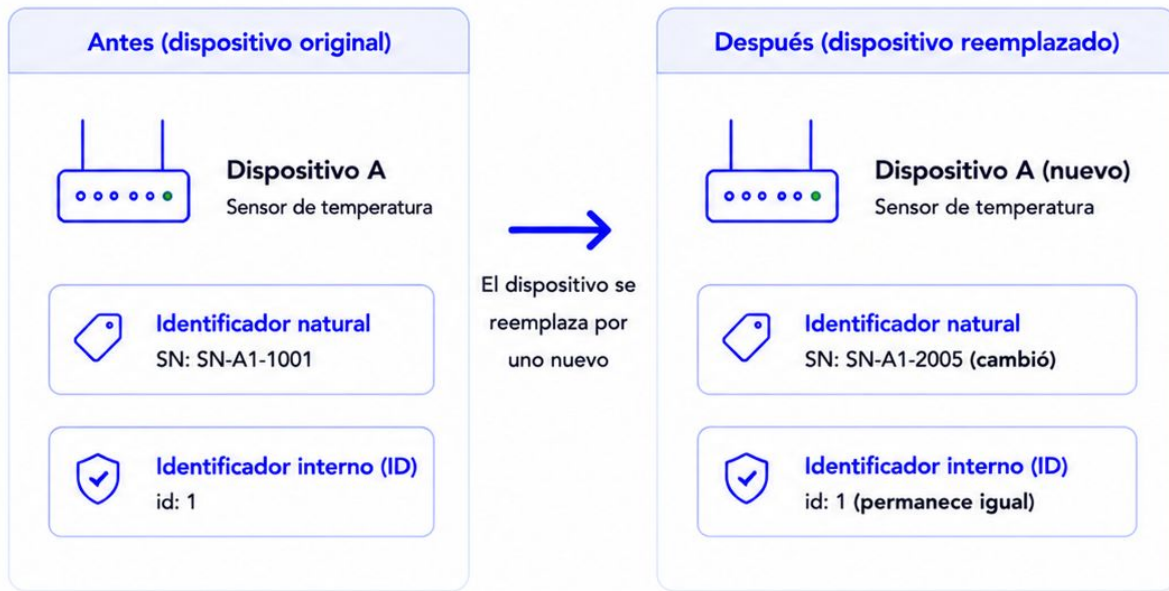
El mundo real puede cambiar

Una clave natural puede:

- cambiar;
- tener errores;
- no estar disponible;
- no ser realmente única;
- depender de otro sistema.

Ejemplo:

un dispositivo reemplazado podría conservar un código interno de inventario.



Que un dato sea único hoy no significa necesariamente que sea un buen identificador interno.

Clave sustituta

Un identificador creado por el sistema

Ejemplo:

`dispositivo_id = 1042`

No describe al dispositivo.

Solo permite identificarlo de forma estable dentro del sistema.



Una clave sustituta separa la identidad interna de los atributos del dominio.

¿Qué clave elegir?

No existe una respuesta universal

Podemos utilizar:

Clave sustituta

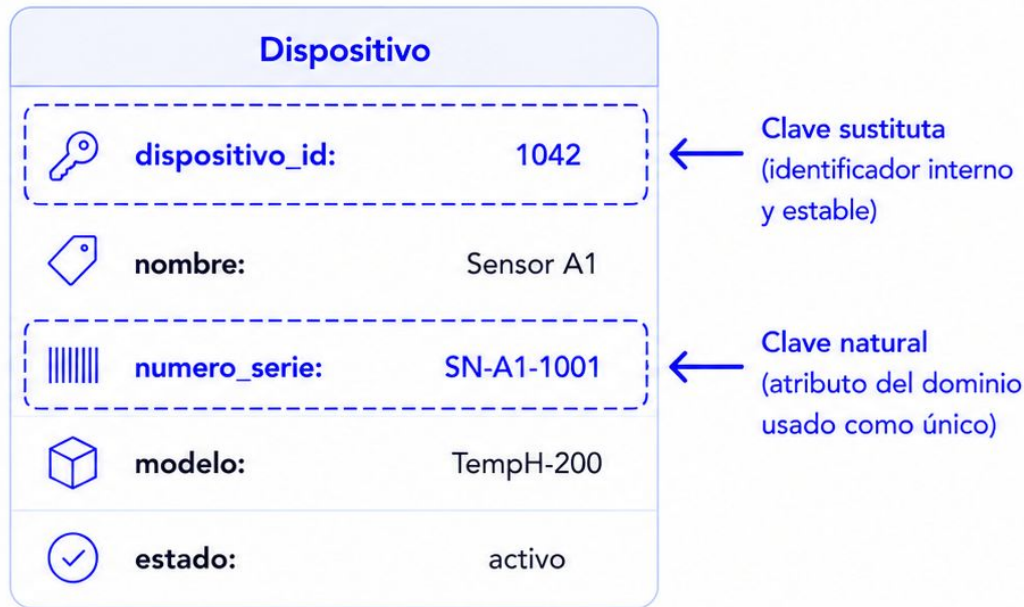
dispositivo_id

y conservar además:

Clave natural

numero_serie

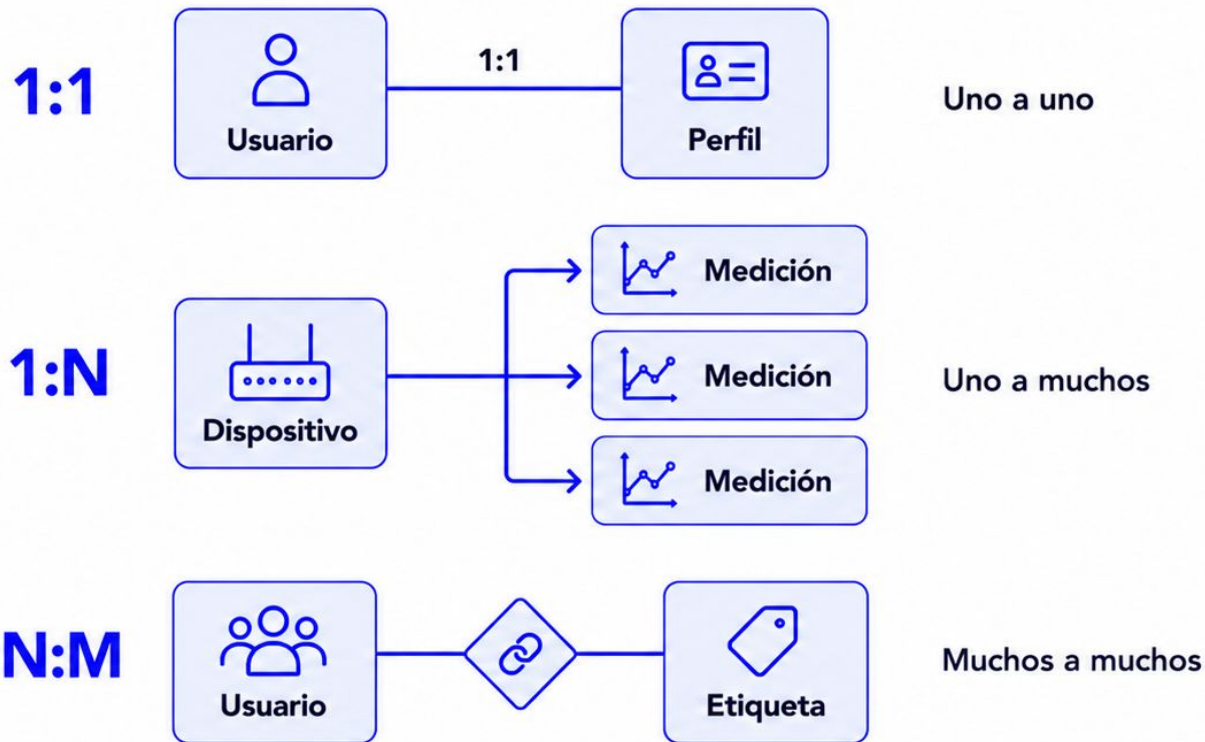
como atributo único.



Identificar una entidad y representar sus datos son decisiones relacionadas, pero diferentes.

5

Relaciones y Cardinalidades

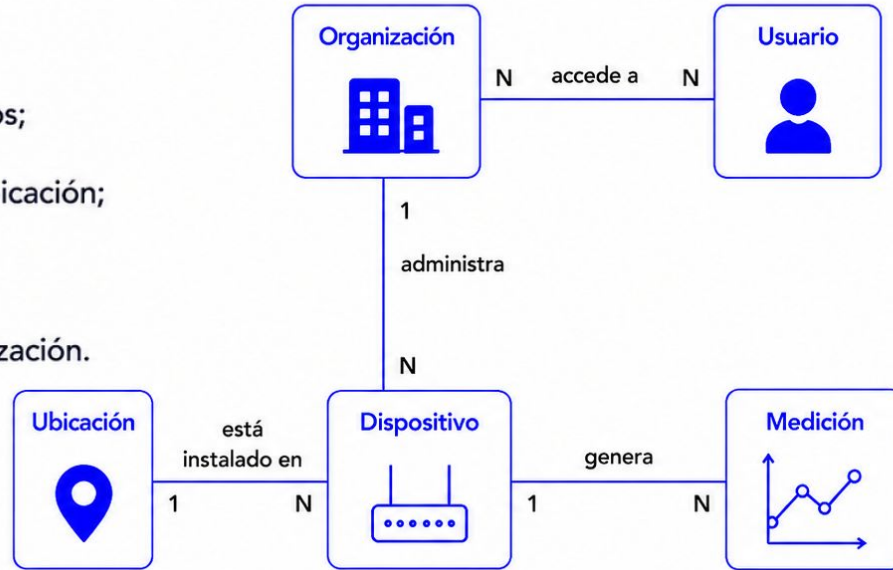


¿Qué es una relación?

El vínculo entre entidades

En nuestro sistema:

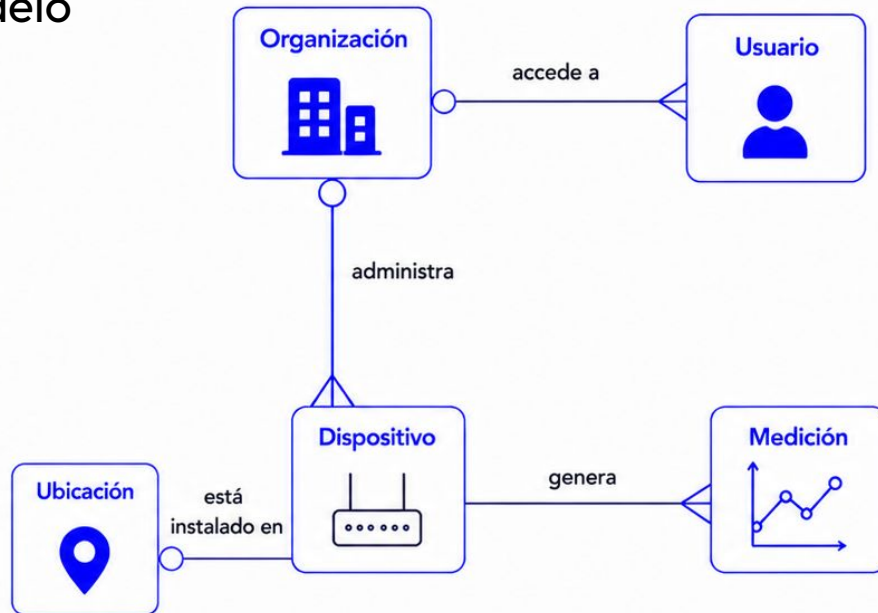
- una **organización** administra dispositivos;
- un **dispositivo** está instalado en una ubicación;
- un **dispositivo** genera mediciones;
- un **usuario** puede acceder a una organización.



Las entidades rara vez tienen sentido completamente aisladas.

Nombrar la relación

Los verbos ayudan a comprender el modelo



Una relación debería poder explicarse mediante una frase del dominio.

Cardinalidad

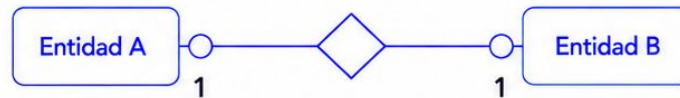
¿Cuántas entidades pueden relacionarse?

La cardinalidad indica cuántas instancias de una entidad pueden vincularse con otra.

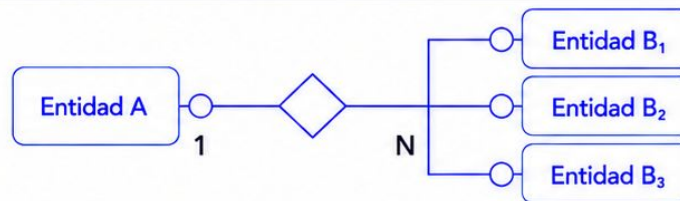
Las principales son:

- uno a uno;
- uno a muchos;
- muchos a muchos.

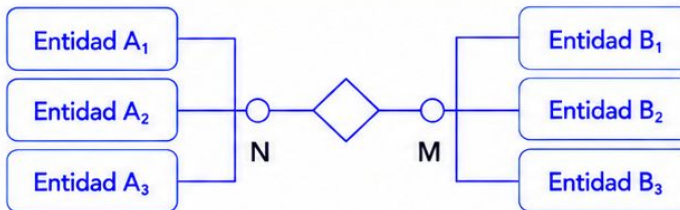
1:1
uno a uno



1:N
uno a muchos



N:M
muchos a muchos



La cardinalidad transforma una regla del dominio en una decisión de diseño.

Relación 1:1

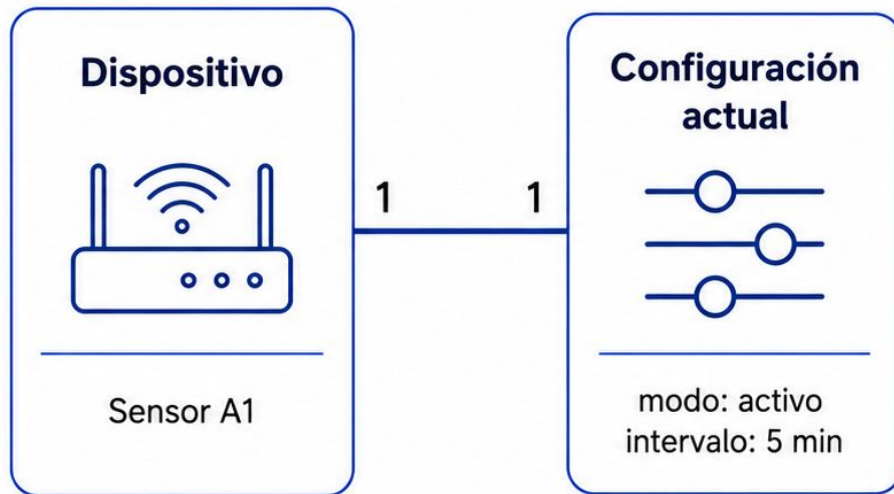
Uno de cada lado

Ejemplo posible:

Dispositivo — configuración_actual

Cada dispositivo tiene una única configuración activa.

Cada configuración activa corresponde a un único dispositivo.



Las relaciones 1:1 existen, aunque son menos frecuentes.

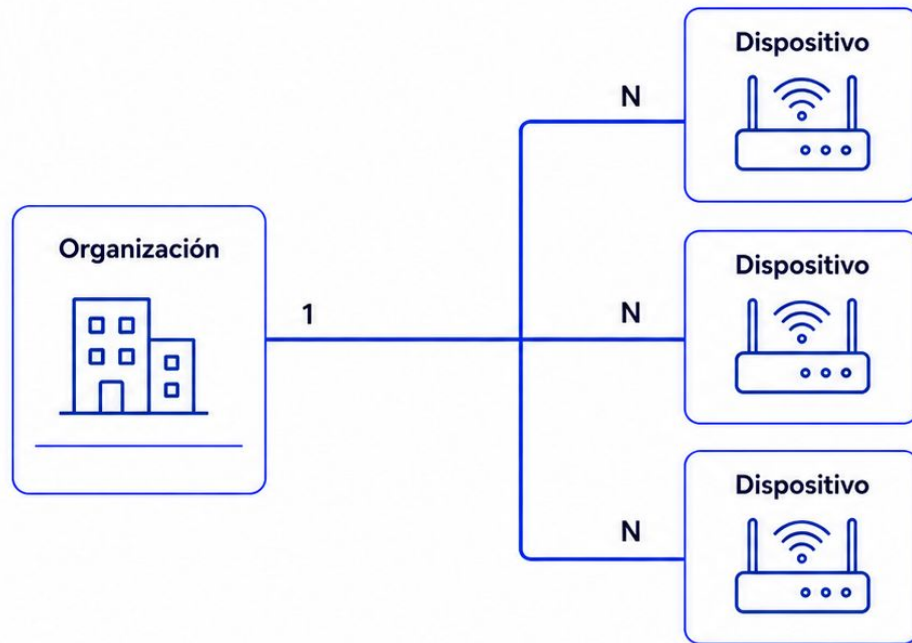
Relación 1:N

Una de las relaciones más frecuentes

Organización → Dispositivos

Una organización puede administrar varios dispositivos.

Cada dispositivo pertenece a una organización.



Uno de un lado puede estar relacionado con muchos del otro.

Dispositivo y mediciones

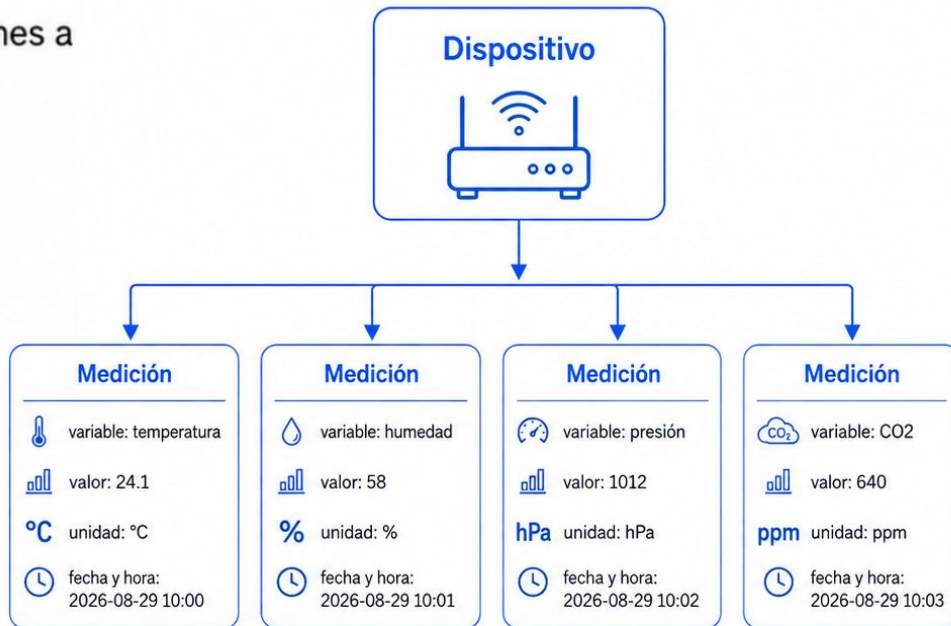
Una relación característica de IoT

Un dispositivo genera muchas mediciones a lo largo del tiempo.

Cada medición corresponde a un único dispositivo.

Cada medición puede indicar, además:

- variable medida;
- valor;
- unidad;
- fecha y hora.



El dispositivo permanece. Las mediciones crecen continuamente.

Relación N:M

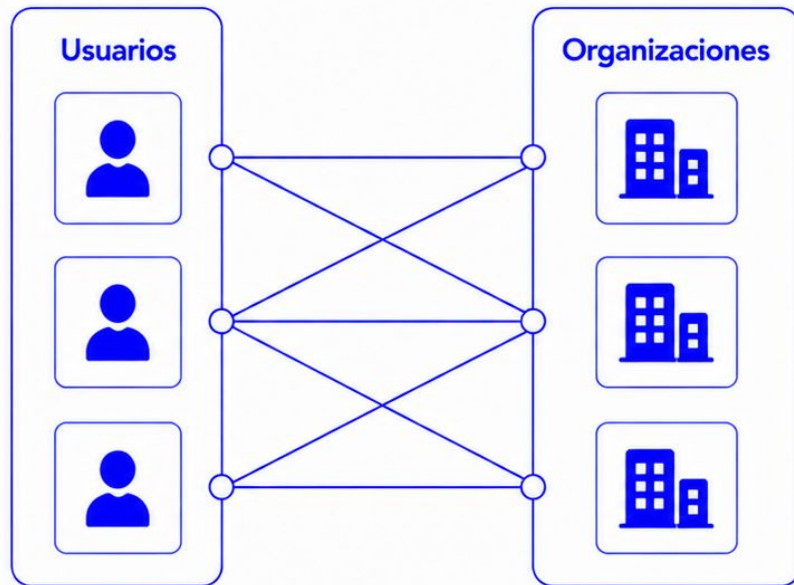
Muchos de ambos lados

Supongamos que:

- un **usuario** puede acceder a varias organizaciones;
- una **organización** puede tener varios usuarios.

Tenemos:

Usuario N:M Organización



Las relaciones muchos a muchos requieren una representación intermedia.

Resolver una relación N:M

La relación también puede tener información propia

Supongamos que:

- un **usuario** puede acceder a varias **organizaciones**;
- una **organización** puede tener varios **usuarios**.

Tenemos:

Usuario N:M Organización



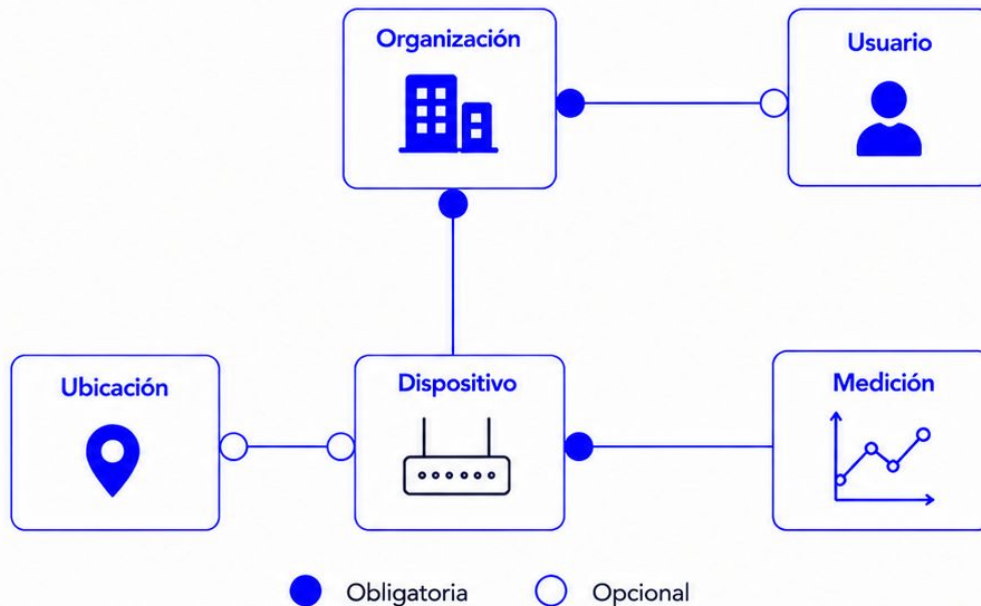
A veces necesitamos guardar información sobre la propia relación.

¿La relación debe existir siempre?

Participación obligatoria u opcional

Ejemplos:

- Un dispositivo **debe** pertenecer a una organización.
- Una medición **debe** pertenecer a un dispositivo.
- Un dispositivo **puede** no tener todavía una ubicación asignada.
- Un usuario **puede** no estar asociado todavía a una organización.



La obligatoriedad también representa reglas del dominio.

6

Restricciones

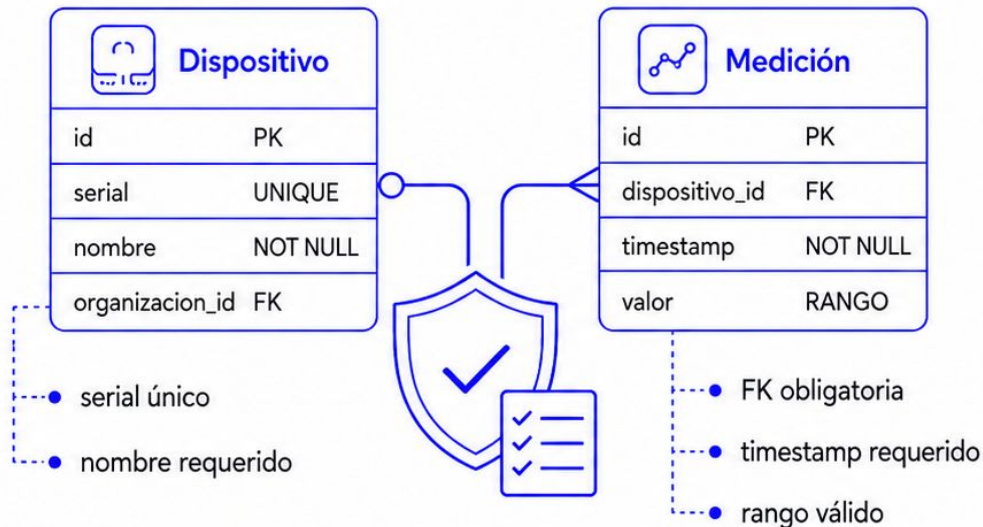


¿Qué es una restricción?

Reglas que deben cumplirse

Ejemplos:

- un número de serie no debería repetirse;
- una medición debe pertenecer a un dispositivo;
- una organización debe tener un nombre;
- un timestamp de medición debe existir;
- determinados valores deben respetar rangos válidos.



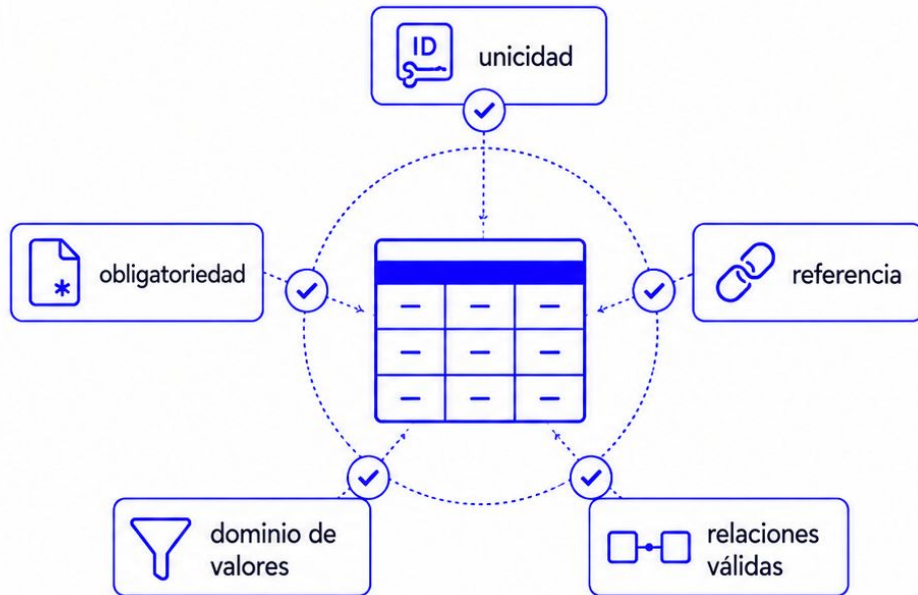
El modelo también representa qué datos no deberían permitirse.

Reglas frecuentes

Protegiendo la consistencia

Podemos necesitar reglas de:

- unicidad;
- obligatoriedad;
- referencia;
- dominio de valores;
- relaciones válidas.



No todas las reglas deberían depender únicamente de la aplicación.

La tecnología no conoce el negocio

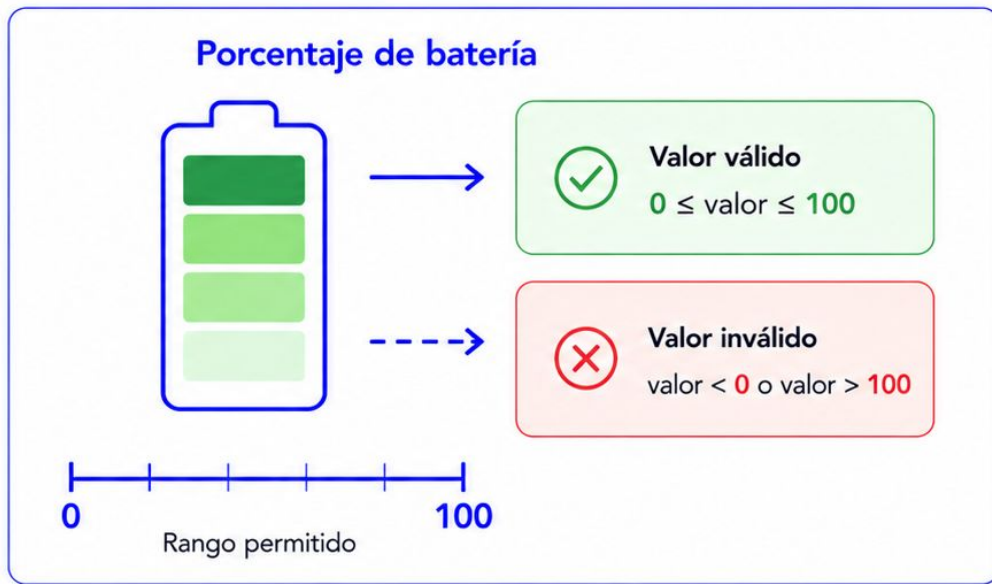
Las reglas nacen del problema

Ejemplo:

Un porcentaje de batería debería encontrarse entre:
0 y 100.

La base de datos no puede inferir esa regla por sí sola.

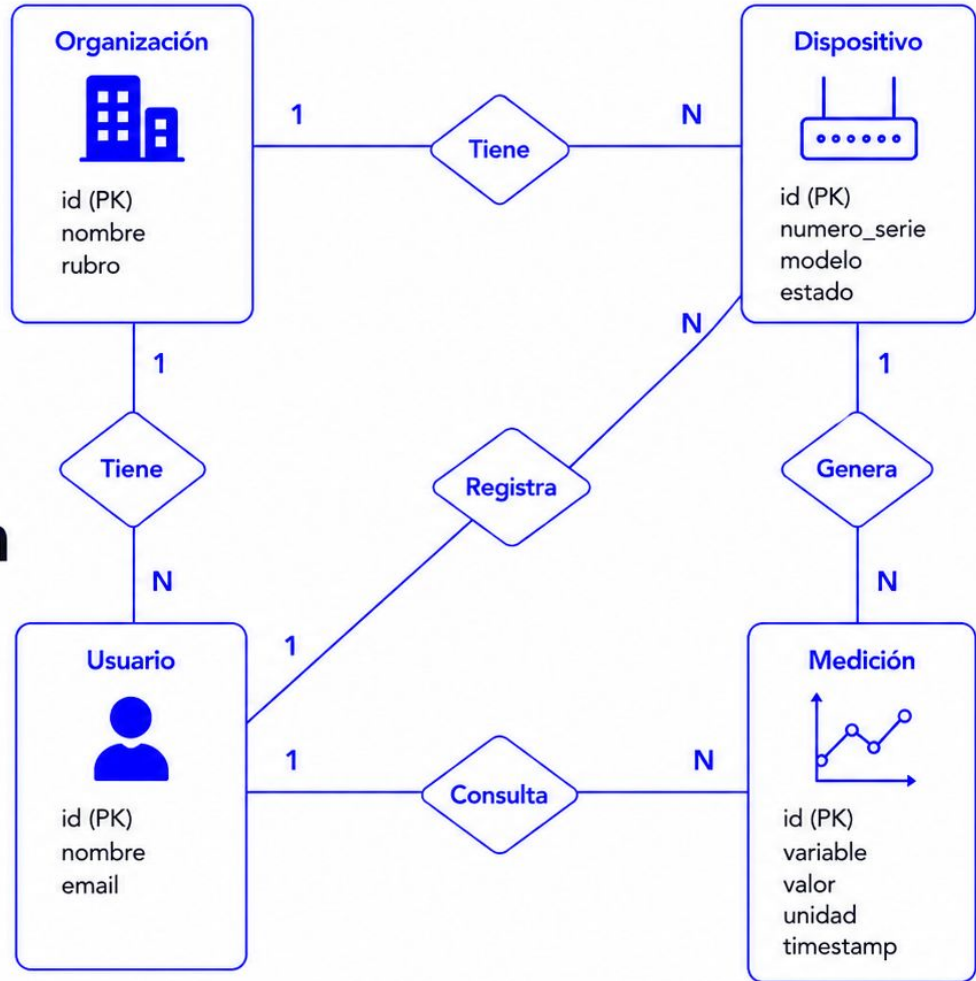
Debemos incorporarla al diseño.



Comprender el dominio permite definir mejores restricciones.

7

Modelo Entidad-Relación

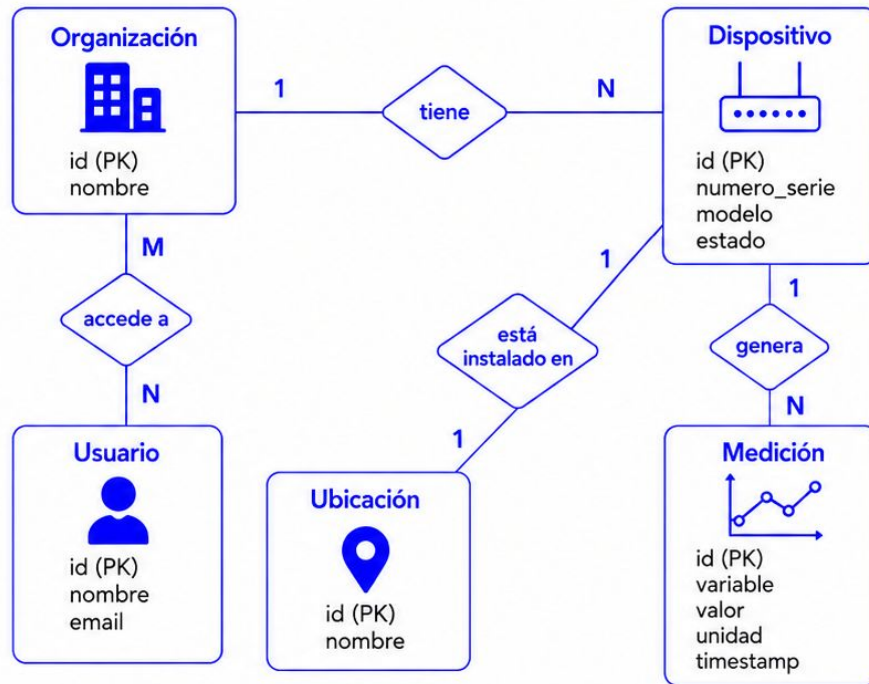


Modelo Entidad-Relación

Representar visualmente el dominio

Permite representar:

- entidades;
- atributos;
- relaciones;
- cardinalidades;
- restricciones.



Un diagrama permite discutir el diseño antes de implementarlo.

Caso guía

Identificamos las entidades

Para nuestro sistema
identificamos inicialmente:

- Organización
- Usuario
- Ubicación
- Dispositivo
- Medición

Organización



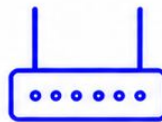
Usuario



Ubicación



Dispositivo



Medición

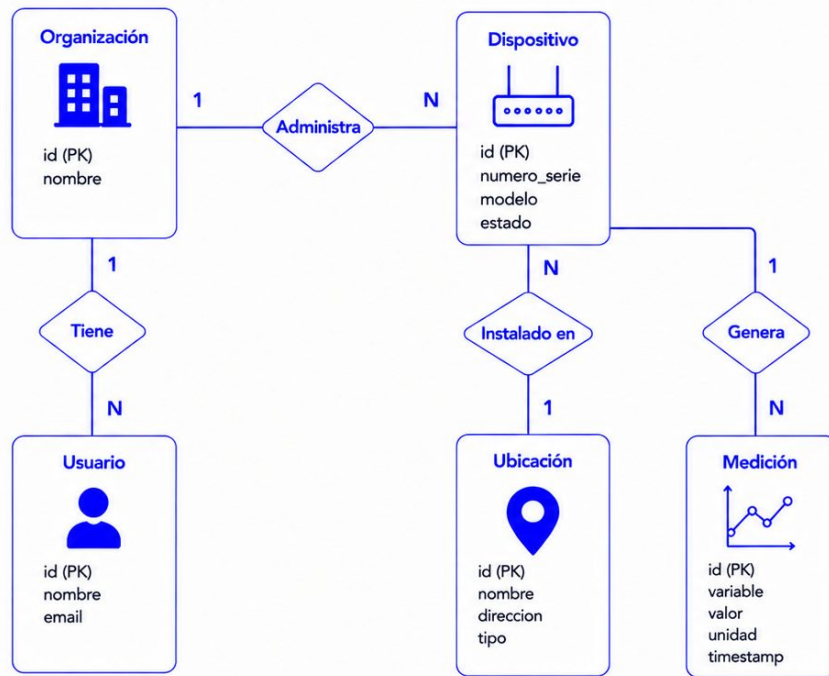


Primero identificamos los conceptos principales del dominio.

Caso guía

Ahora contamos la historia del sistema

- Una organización administra dispositivos.
- Una organización tiene usuarios.
- Un dispositivo puede estar instalado en una ubicación.
- Un dispositivo genera mediciones.

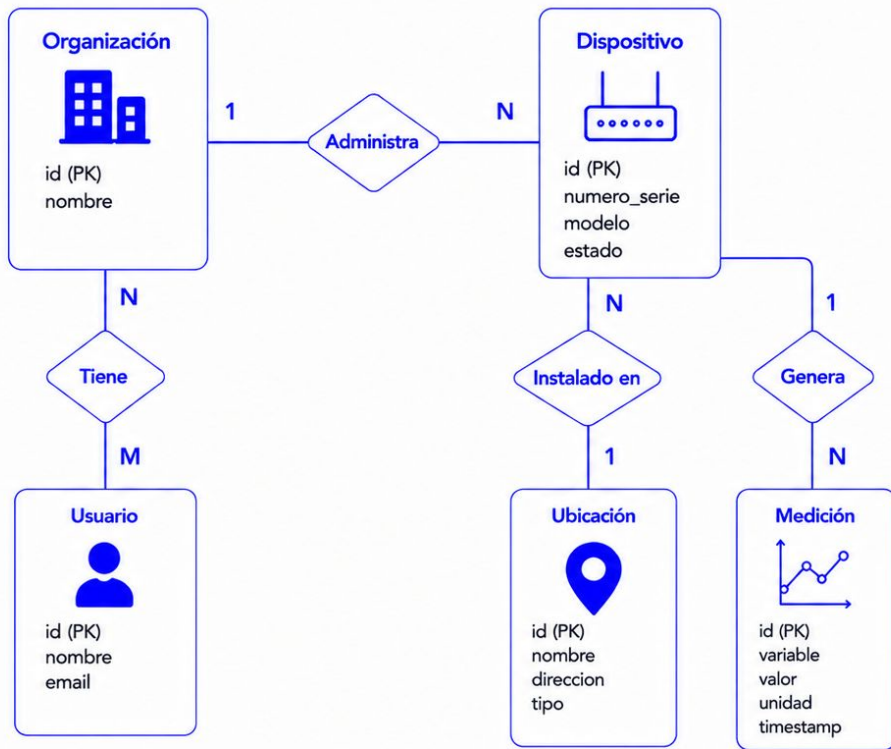


Las relaciones explican cómo interactúan las entidades del dominio.

Caso guía

Definimos cuántos

- Organización 1:N Dispositivo
- Organización N:M Usuario
- Ubicación 1:N Dispositivo
- Dispositivo 1:N Medición



Ahora el modelo empieza a contener reglas concretas.

Construir el modelo

No intentamos resolver todo de una vez

1 Paso 1

Dispositivo

2 Paso 2

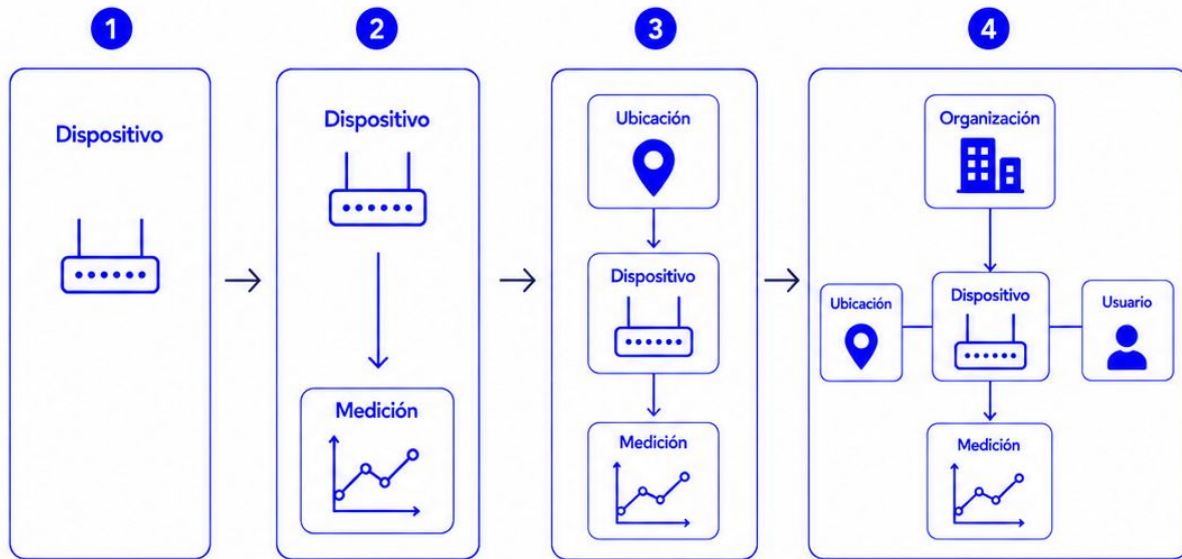
Dispositivo → Medición

3 Paso 3

Ubicación → Dispositivo → Medición

4 Paso 4

Incorporamos organización y usuarios alrededor de la estructura anterior.



Un modelo puede construirse y validarse progresivamente.

Del modelo conceptual al lógico

Incorporar estructura

Cada entidad empieza a incorporar:

- identificador;
- atributos;
- relaciones;
- reglas.



Entidad conceptual



Medición	
	medicion_id
	dispositivo_id
	variable
123	valor
A ^b C	unidad
	timestamp

Modelo lógico (estructura)



Todavía pensamos el modelo. La implementación vendrá después.

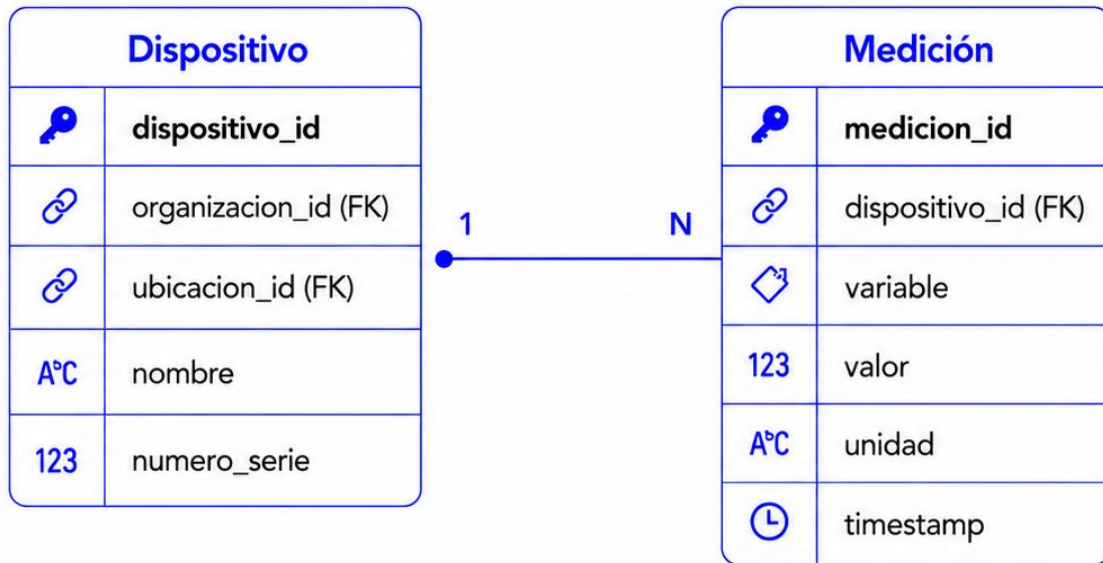
Modelo lógico inicial

El caso guía empieza a tomar estructura

Las entidades del caso ahora incorporan identificadores y referencias entre sí.

Por ejemplo:

- dispositivo se relaciona con organización y ubicación
- medición se relaciona con dispositivo

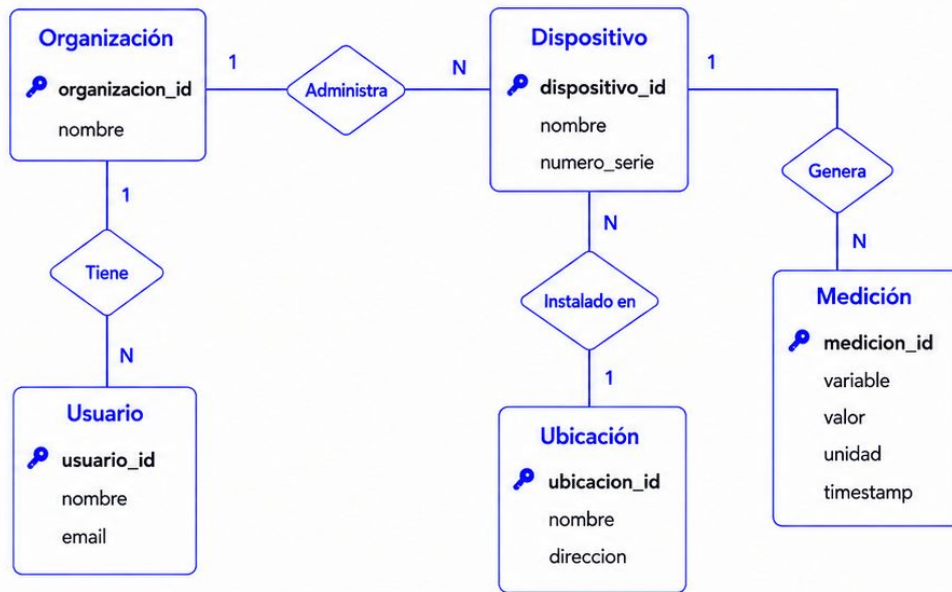


El modelo conceptual comienza a transformarse en una estructura de datos.

Validar antes de implementar

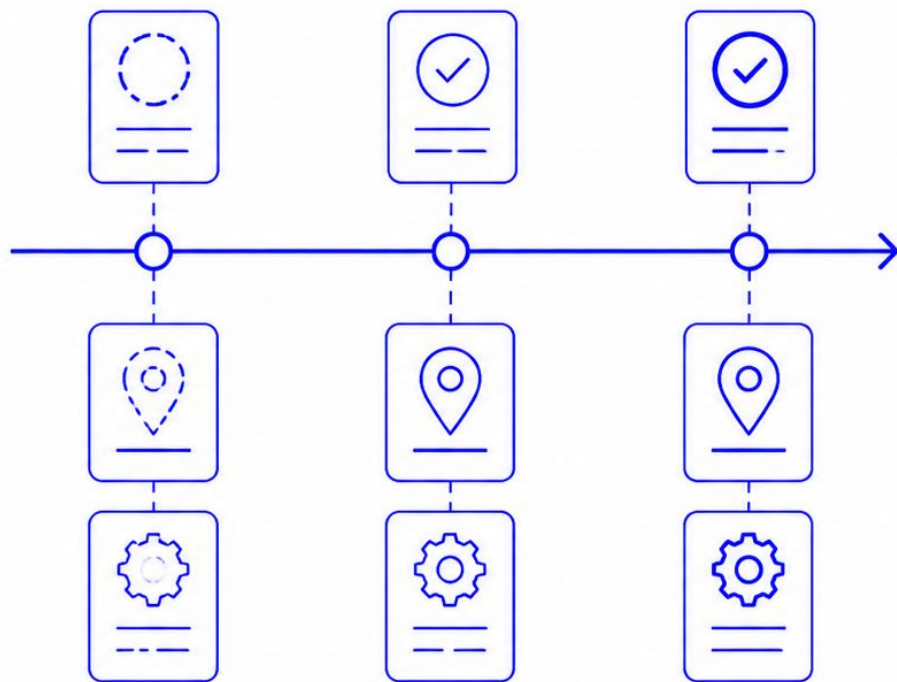
Preguntas para revisar el diseño

- ✓ ¿Cada entidad representa algo necesario?
- ✓ ¿Los atributos están donde corresponden?
- ✓ ¿Las relaciones representan el dominio?
- ✓ ¿Las cardinalidades son correctas?
- ✓ ¿Hay datos duplicados innecesariamente?
- ✓ ¿Faltan restricciones?
- ✓ ¿Podemos responder las preguntas que necesita el sistema?



Corregir el modelo antes de implementar suele ser mucho más sencillo.

Estados e Historiales



Estados e historiales

Ahora necesitamos decidir cómo representarlos

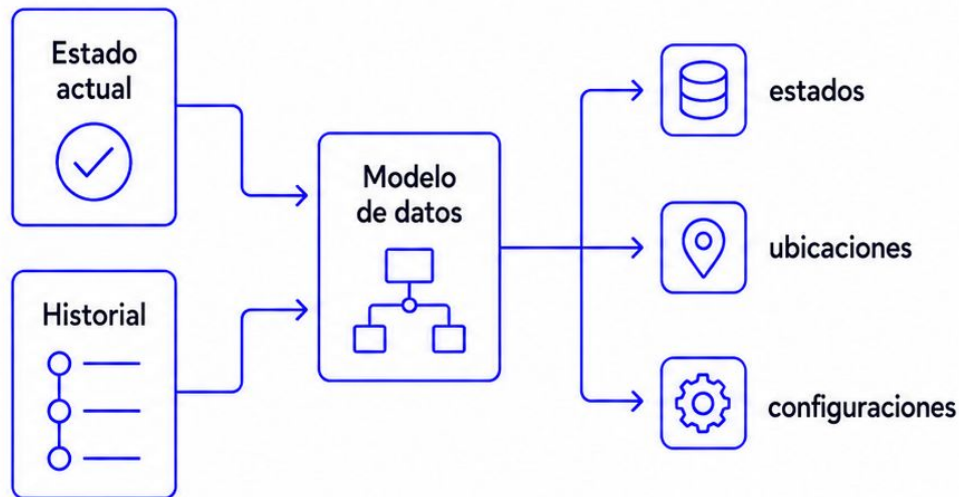
En la Clase 1 vimos que un sistema puede necesitar conocer tanto el estado actual como su evolución histórica.

Hoy la pregunta es otra:

¿Cómo representamos esa necesidad dentro del modelo de datos?

Vamos a analizar tres casos:

- historial de estados;
- cambios de ubicación;
- versiones de configuración.



El concepto ya lo conocemos. Ahora debemos convertirlo en estructura.

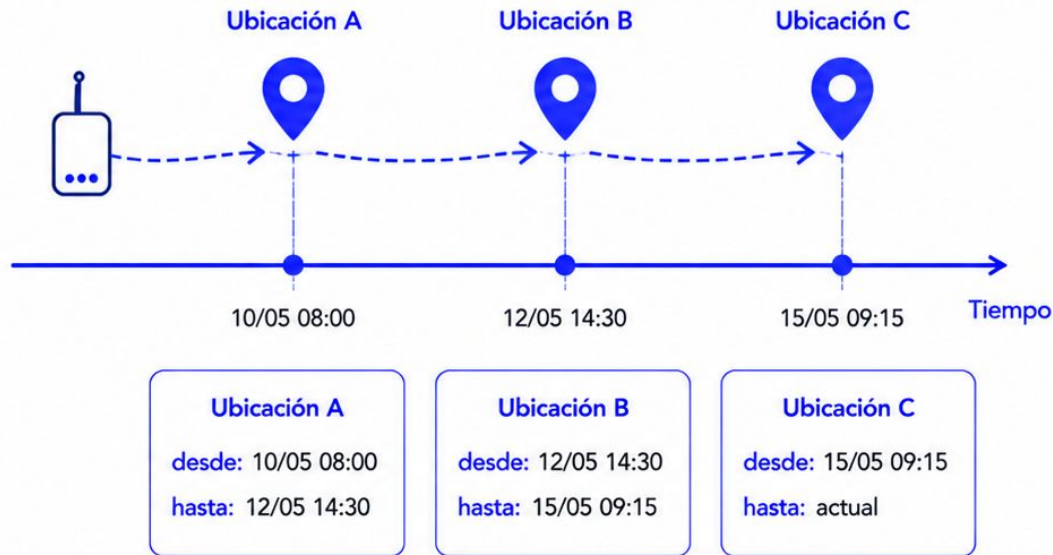
Historial de ubicación

Cuando una relación también cambia

Si un dispositivo puede trasladarse, guardar solamente su ubicación actual no alcanza cuando necesitamos reconstruir movimientos anteriores.

Podemos modelar una relación histórica que indique:

- dispositivo;
- ubicación;
- desde;
- hasta.



También podemos necesitar historizar relaciones entre entidades.

Configuraciones

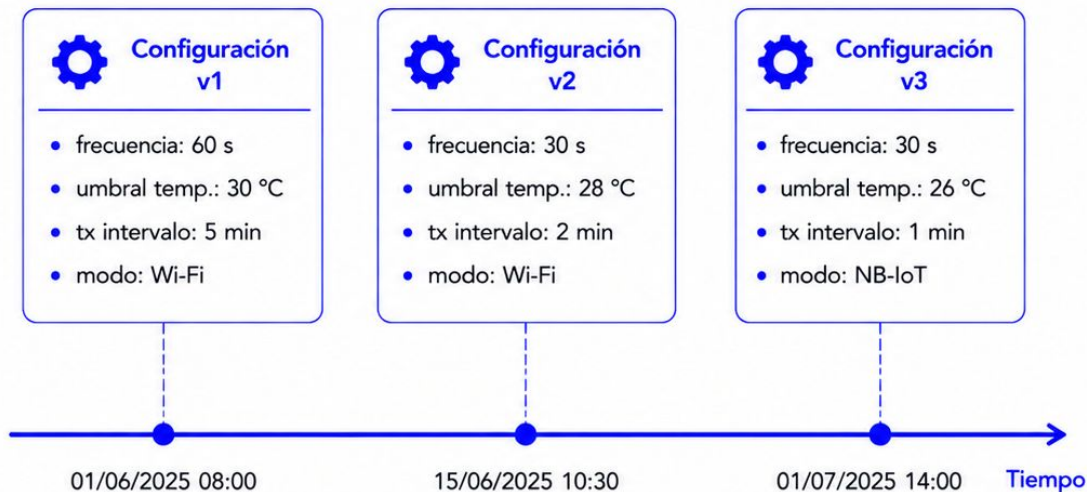
¿Sobrescribir o versionar?

Una configuración puede incluir:

- frecuencia de muestreo;
- umbrales;
- intervalos de transmisión;
- parámetros de comunicación.

Si simplemente actualizamos sus valores, perdemos la configuración anterior.

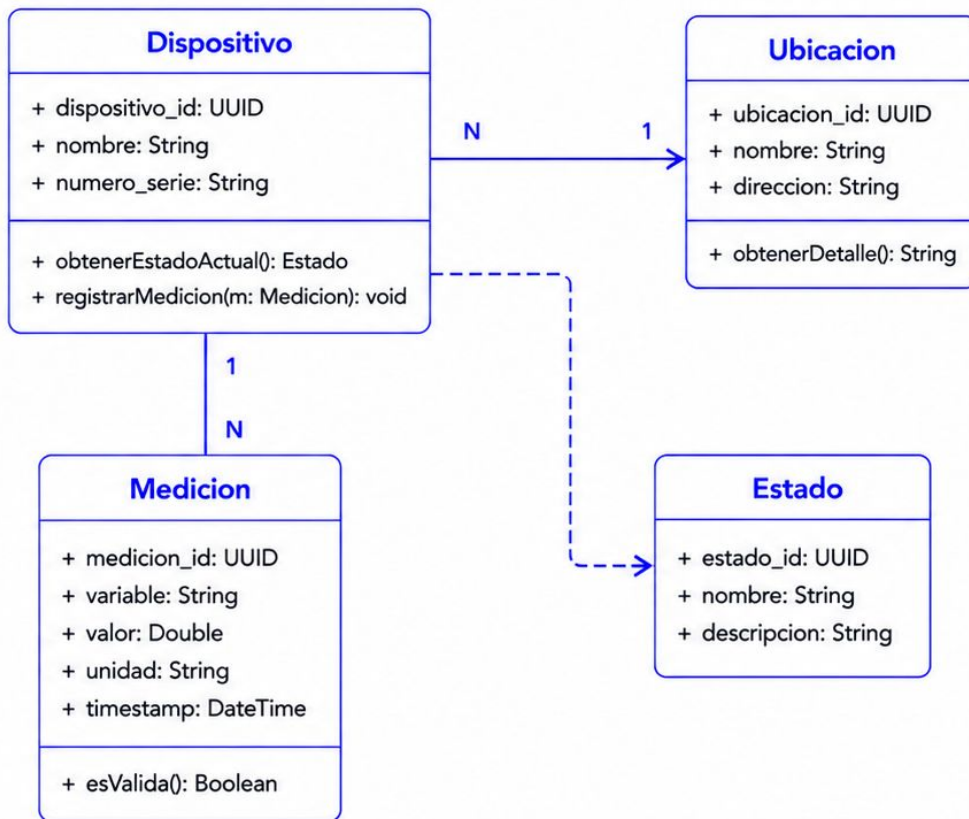
Una alternativa es representar distintas versiones y registrar cuándo estuvo vigente cada una.



Antes de sobrescribir un dato debemos preguntarnos si necesitamos conservar sus versiones anteriores.

9

UML aplicado al Modelado de Datos



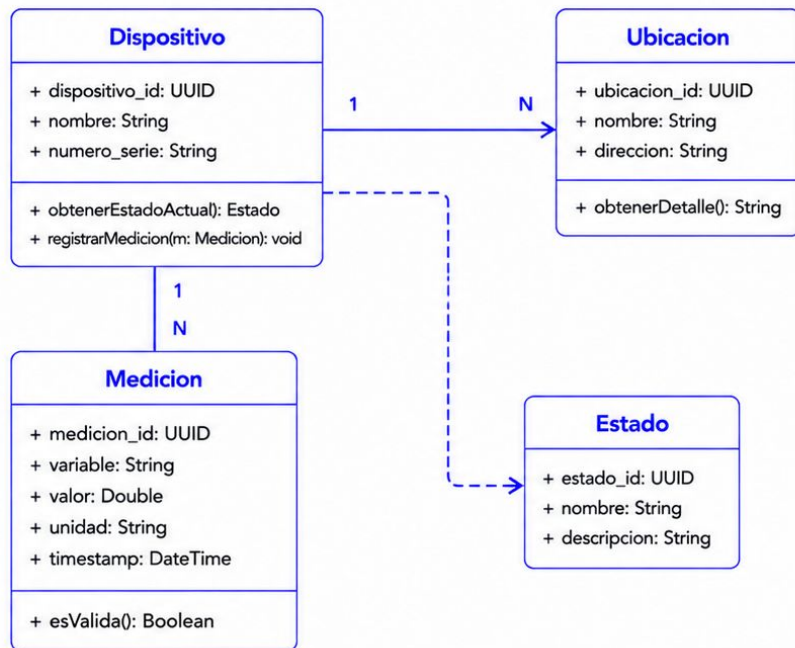
UML

Otra forma de representar estructura

Los diagramas de clases permiten representar:

- clases;
- atributos;
- asociaciones;
- multiplicidades.

Pueden utilizarse como herramienta para analizar estructuras de datos.



UML y ER utilizan notaciones diferentes para representar conceptos similares.

Clase UML y entidad

Conceptos cercanos, pero no idénticos

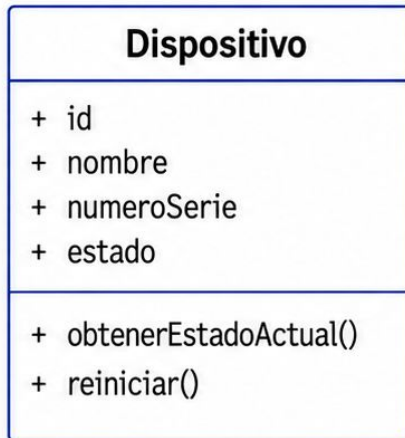
Dispositivo

- id
- nombre
- numeroSerie
- estado

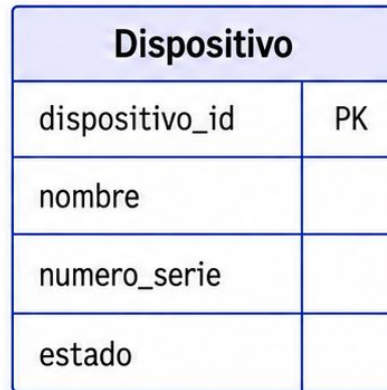
Una clase puede representar comportamiento.

Una entidad pone el foco en la información persistente.

Clase UML



Entidad de datos



Un modelo de software y un modelo de datos pueden estar relacionados sin ser iguales.

Multiplicidades UML

Representar cardinalidades

En UML, las multiplicidades indican cuántas instancias de una clase pueden asociarse con otra.

Por ejemplo:

- Organización 1 — 0..* Dispositivo
- Dispositivo 1 — 0..* Medición



Las multiplicidades también expresan cuántas instancias pueden relacionarse.

ER vs UML

Dos herramientas para modelar

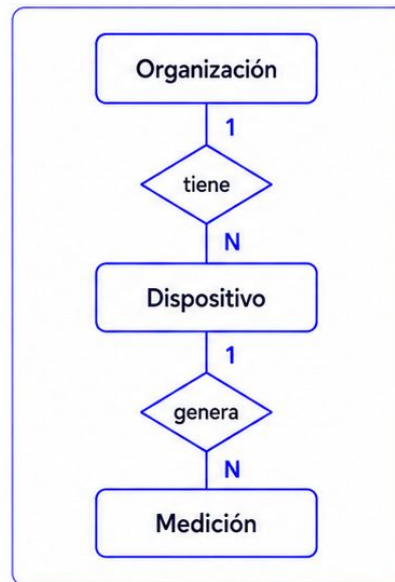
Entidad-Relación

- orientado al modelado de datos;
- entidades;
- relaciones;
- cardinalidades.

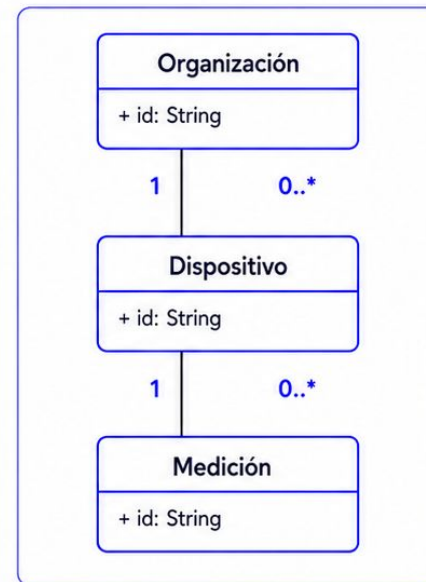
UML

- utilizado ampliamente en diseño de software;
- clases;
- asociaciones;
- multiplicidades.

ER



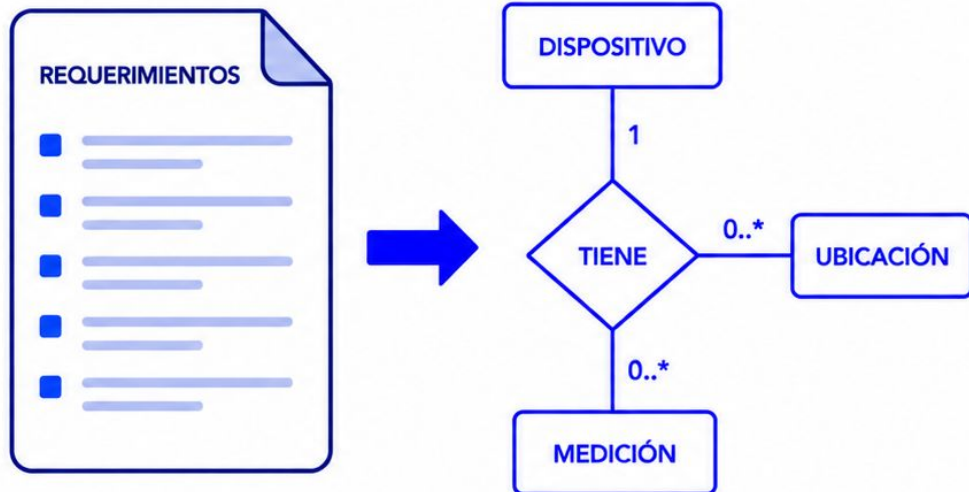
UML



La notación es una herramienta. Lo importante es representar correctamente el dominio.

10

De los Requerimientos al Modelo



Un requerimiento

Leamos antes de modelar



“Cada **organización** puede administrar distintos **dispositivos** instalados en diferentes **ubicaciones**. Cada dispositivo registra periódicamente distintas **variables ambientales** y las **mediciones** deben conservarse históricamente.”



Los requerimientos contienen pistas sobre entidades, relaciones y reglas.

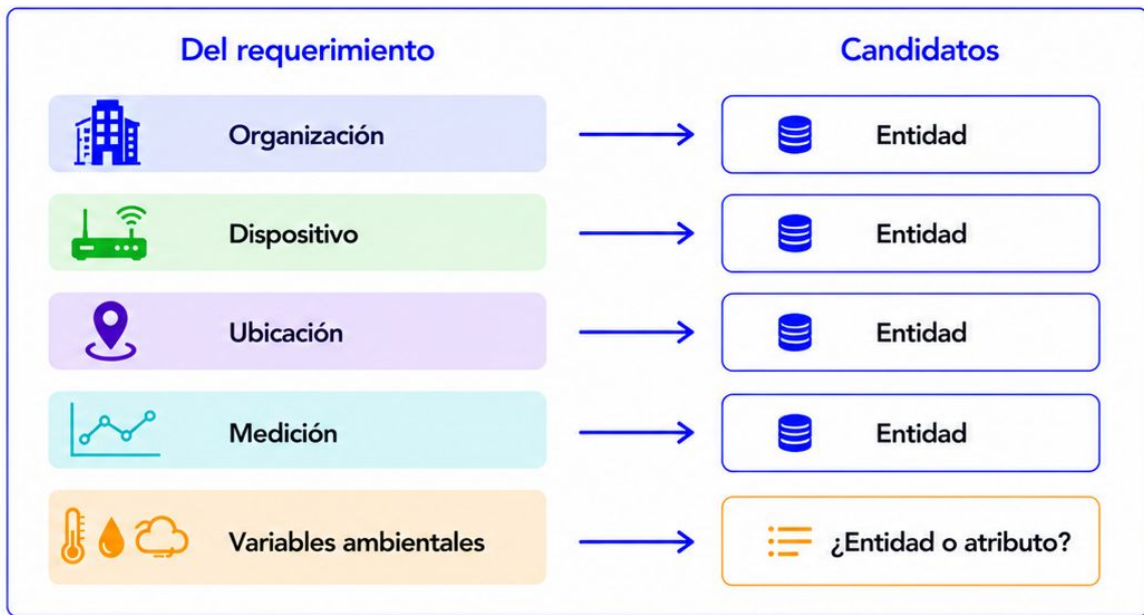
Identificar sustantivos

Posibles entidades

Del requerimiento aparecen:

- Organización
- Dispositivo
- Ubicación
- Medición

También aparecen conceptos como *variables ambientales*, que debemos analizar para decidir si serán entidades o atributos.



Los sustantivos suelen ser buenos candidatos a entidades, pero debemos validar si realmente necesitamos representarlos.

Identificar verbos

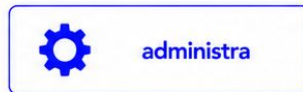
Posibles relaciones

Una organización **administra** dispositivos.

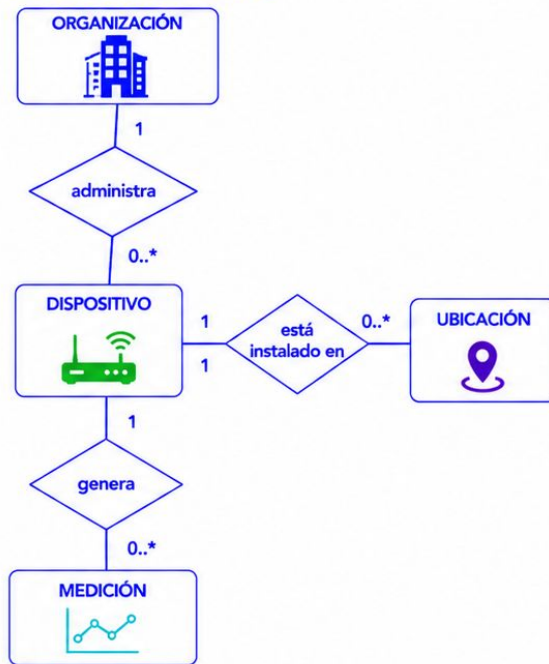
Un dispositivo **está instalado en** una ubicación.

Un dispositivo **genera** mediciones.

Verbos del requerimiento



Relaciones entre entidades



Los verbos ayudan a descubrir relaciones.

Identificar restricciones

El requerimiento también dice cómo funciona el sistema

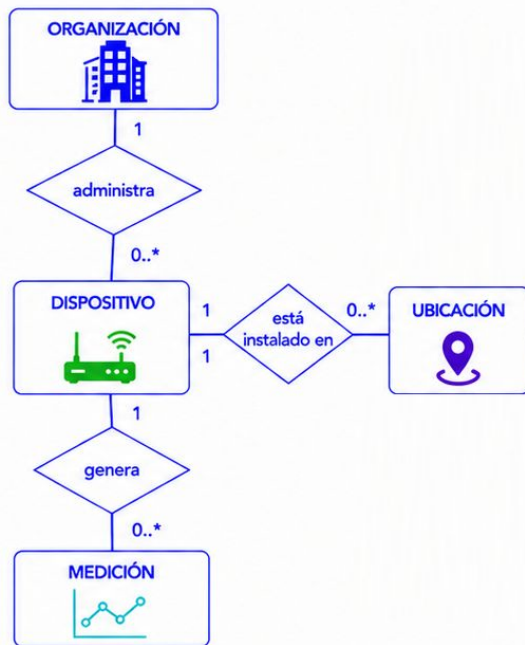
“Cada dispositivo registra periódicamente distintas variables ambientales.”

Esto implica:

- cada medición pertenece a un dispositivo;
- una medición representa una variable determinada;
- cada medición debe indicar cuándo fue registrada.

“Las mediciones deben conservarse históricamente.”

Esto implica que el modelo debe permitir múltiples mediciones del mismo dispositivo a lo largo del tiempo.



Restricciones identificadas



Cada medición pertenece a un dispositivo.

Cardinalidad: Dispositivo 1 — 0..* Medición



Una medición representa una variable determinada.

Atributo en Medición: variable_id



Cada medición debe indicar cuándo fue registrada.

Atributo en Medición: timestamp



Las mediciones se conservan históricamente.

Permitir múltiples mediciones en el tiempo para un mismo dispositivo.



Las reglas del negocio también forman parte del modelo.

El modelo evoluciona

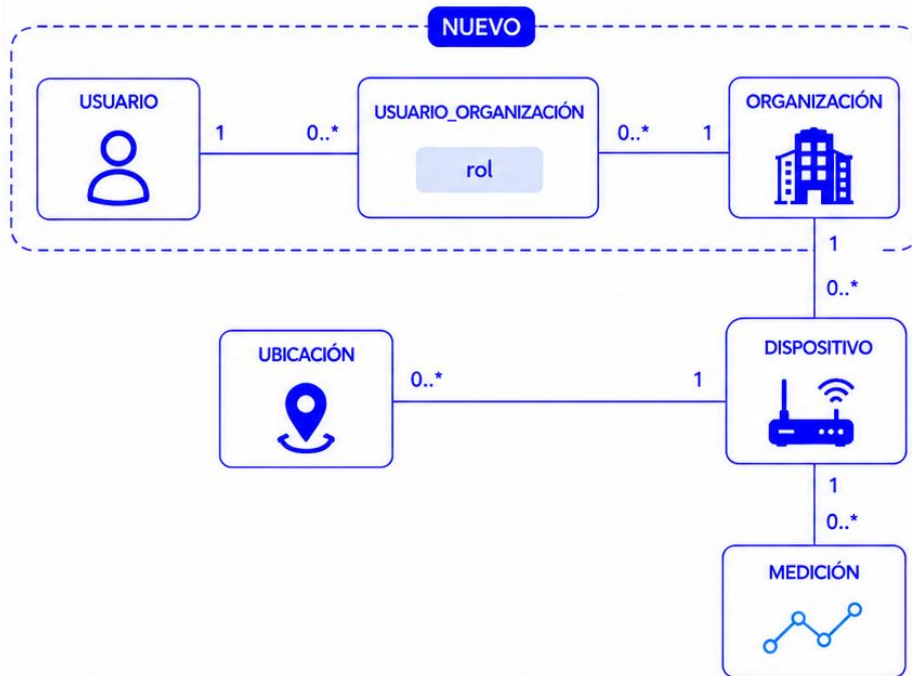
Los requerimientos cambian

Nuevo requerimiento:

“ Los usuarios pueden acceder a una o más organizaciones y tener diferentes roles en cada una.

Ahora necesitamos incorporar:

- Usuario;
- relación Usuario–Organización;
- Rol.

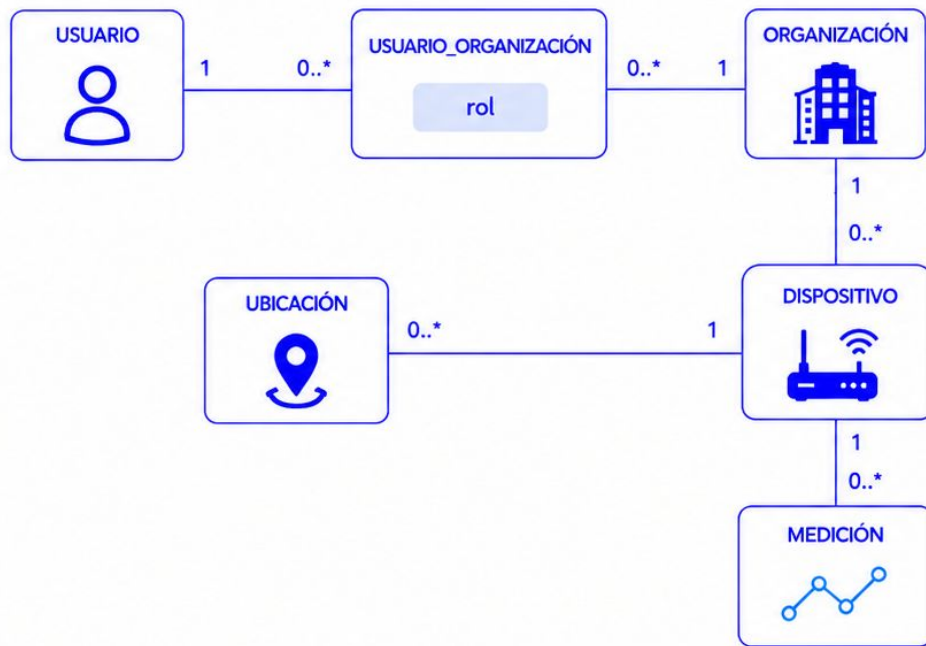


Un modelo no es necesariamente definitivo. Evoluciona junto con los requerimientos.

11

Práctica Guiada

Construcción del modelo de datos de una aplicación IoT.



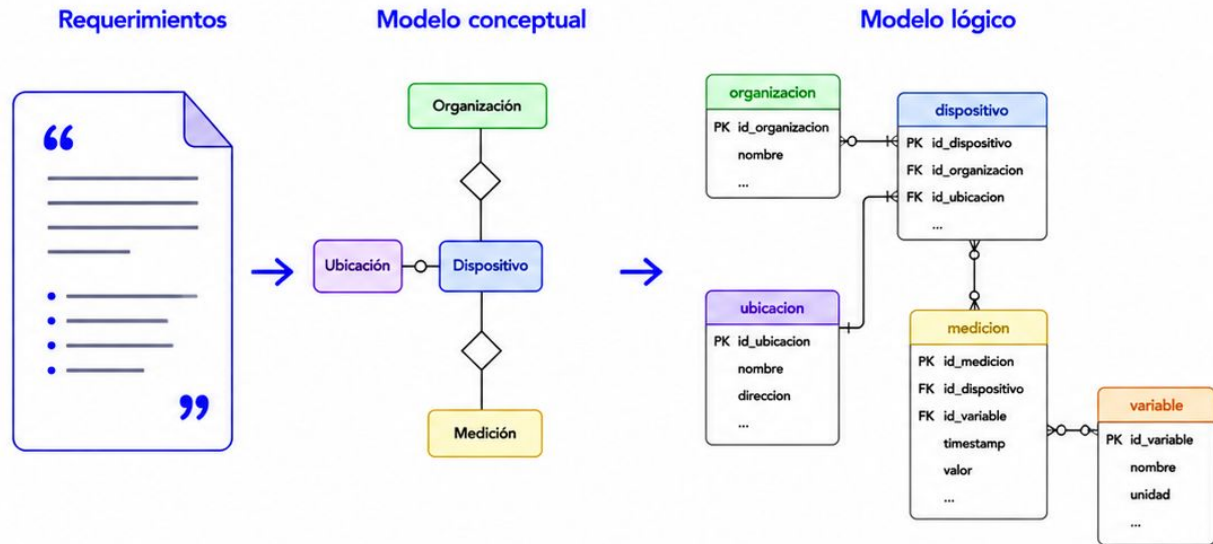
Práctica

Del sistema IoT al modelo de datos

Vamos a tomar el caso guía trabajado durante la clase y transformarlo en un modelo completo.

Vamos a definir:

- entidades y atributos;
- identificadores;
- relaciones;
- cardinalidades;
- restricciones;
- necesidades de historial.



El objetivo es transformar requerimientos en una estructura coherente.

Paso 1

Leer los requerimientos

Vamos a identificar:

- qué elementos existen;
- qué información debemos conservar;
- qué reglas aparecen;
- qué preguntas debería responder el sistema.

Requerimientos del sistema IoT



Una **organización** administra **dispositivos** instalados en distintas ubicaciones.



Cada dispositivo registra periódicamente distintas **variables ambientales**.



Cada dispositivo está instalado en una **ubicación**.



Las mediciones deben **conservarse** **históricamente**.



Reglas del negocio

- ✓ Una medición pertenece a un dispositivo.
- ✓ Una medición representa una variable determinada.
- ✓ Cada medición debe indicar cuándo fue registrada.
- ✓ Debe existir historial de mediciones por dispositivo.



Preguntas que debe responder el sistema

- ¿Qué dispositivos tiene una organización?
- ¿Dónde está instalado cada dispositivo?
- ¿Cuáles son las últimas mediciones de un dispositivo?
- ¿Cuál fue la evolución histórica de una variable?



Antes de dibujar, debemos comprender el problema.

Paso 2

Identificar entidades

Posibles entidades:

- Organización
- Usuario
- Ubicación
- Dispositivo
- Medición

Vamos a discutir:

- cuáles necesitamos;
- cuáles podrían ser atributos;
- si falta alguna.



No todo sustantivo necesita convertirse en una entidad.

Paso 3

Definir atributos

Para cada entidad vamos a definir la información necesaria.

Ejemplo:

Dispositivo

- identificador;
- nombre;
- número de serie;
- modelo;
- fecha de instalación;
- estado.

Medición

- variable;
- valor;
- unidad;
- timestamp.



Dispositivo

- identificador;
- nombre;
- número de serie;
- modelo;
- fecha de instalación;
- estado.



Medición

- variable;
- valor;
- unidad;
- timestamp.



Cada atributo debe tener una razón para existir.

Paso 4

Elegir identificadores

Vamos a analizar:

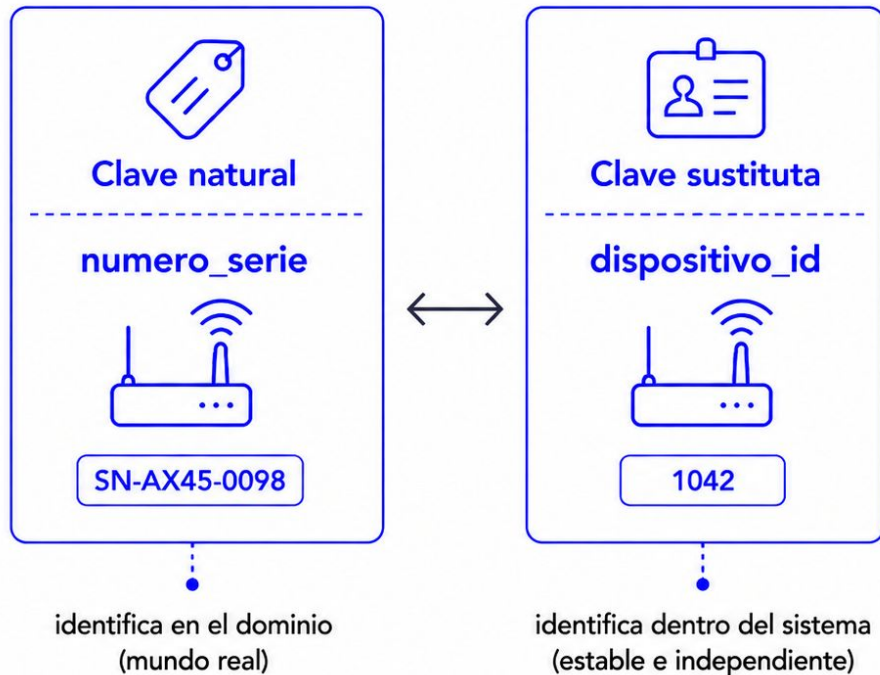
- qué datos son naturalmente únicos;
- cuáles podrían cambiar;
- cuáles deberían utilizarse como identificadores internos.

Compararemos:

numero_serie

con:

dispositivo_id



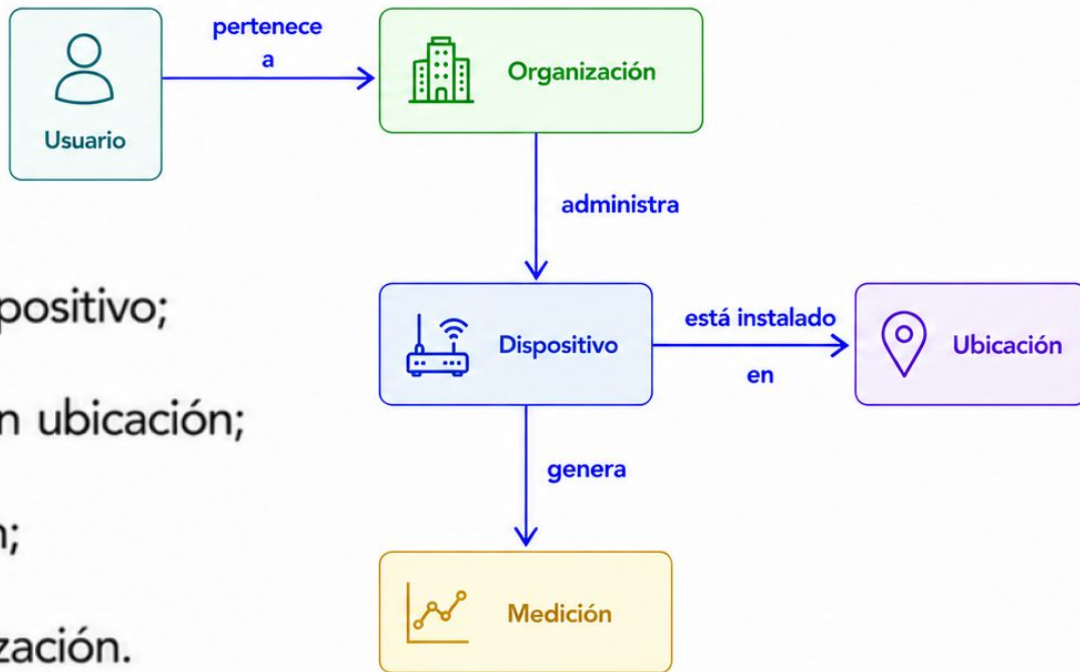
La identidad interna no siempre debe depender de un dato del mundo real.

Paso 5

Identificar relaciones

Vamos a expresar las relaciones mediante frases:

- organización **administra** dispositivo;
- dispositivo **está instalado** en ubicación;
- dispositivo **genera** medición;
- usuario **pertenece** a organización.



Si no podemos explicar claramente una relación, debemos revisar el modelo.

Paso 6

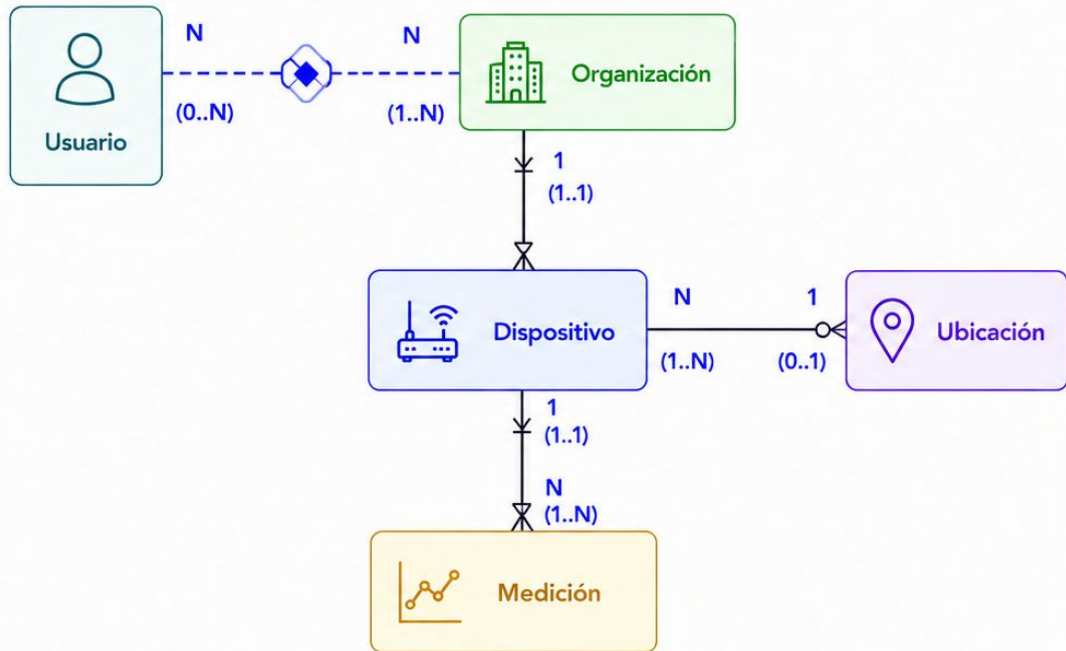
Definir cardinalidades

Vamos a determinar:

- uno a uno;
- uno a muchos;
- muchos a muchos;
- participación obligatoria;
- participación opcional.

Aplicándolo al caso:

- Organización **1:N** Dispositivo;
- Dispositivo **1:N** Medición;
- Usuario **N:M** Organización.



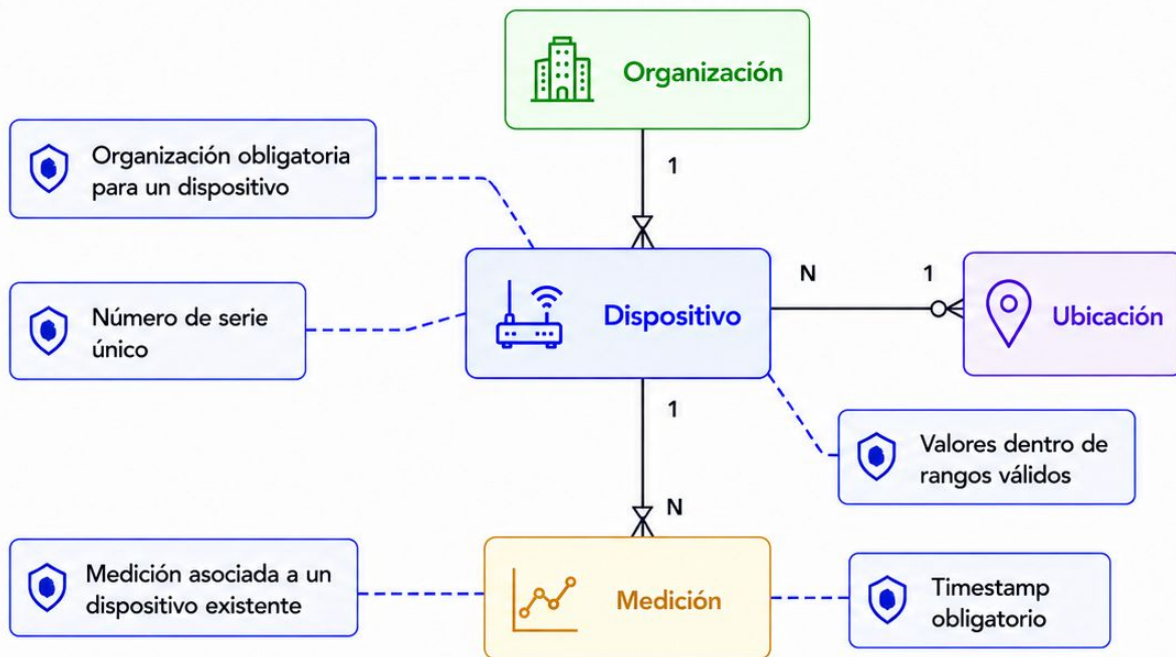
La cardinalidad representa reglas concretas del sistema.

Paso 7

Definir restricciones

Vamos a discutir reglas como:

- número de serie único;
- medición asociada a un dispositivo existente;
- timestamp obligatorio;
- organización obligatoria para un dispositivo;
- valores dentro de rangos válidos.



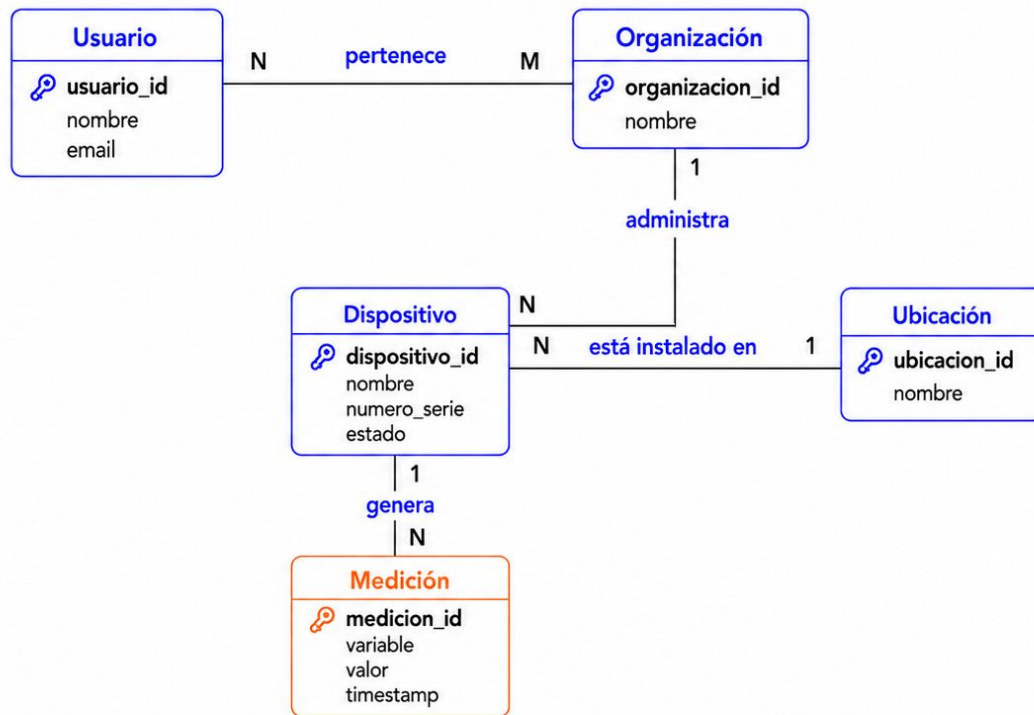
Las restricciones protegen la consistencia del modelo.

Paso 8

Construir el diagrama ER

Vamos a integrar:

- entidades;
- atributos;
- identificadores;
- relaciones;
- cardinalidades.



Ahora las decisiones aparecen juntas en una única representación.

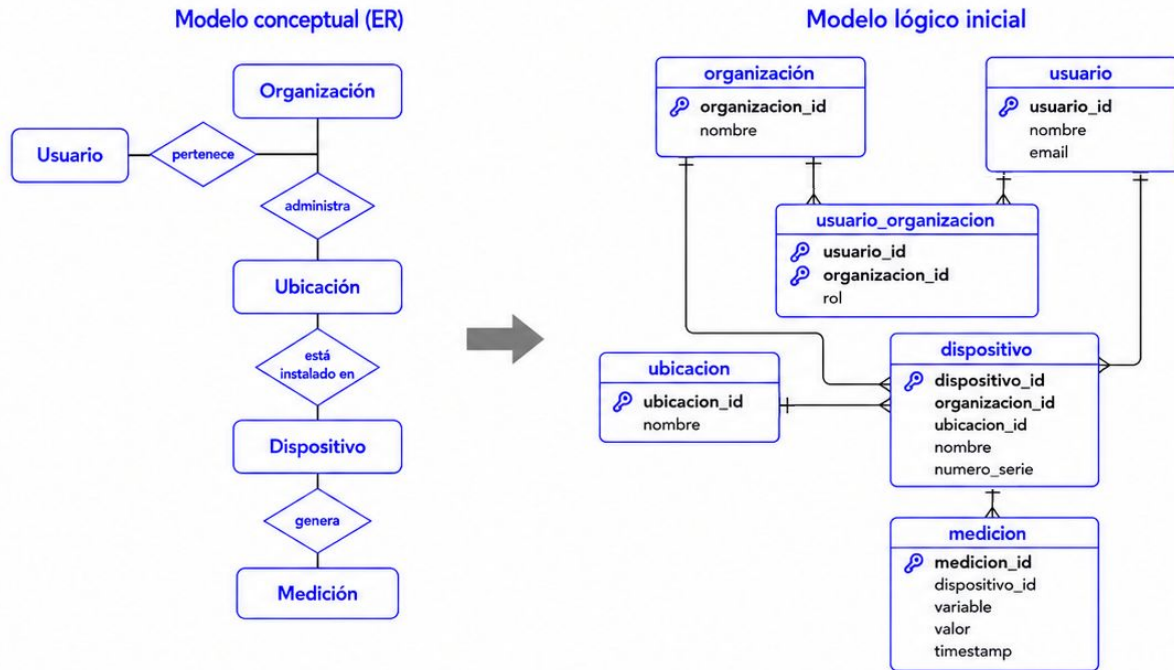
Paso 9

Transformar el modelo conceptual

A partir del ER vamos a obtener una primera estructura lógica.

Vamos a incorporar:

- identificadores;
- referencias entre entidades;
- atributos necesarios;
- entidades asociativas cuando corresponda.



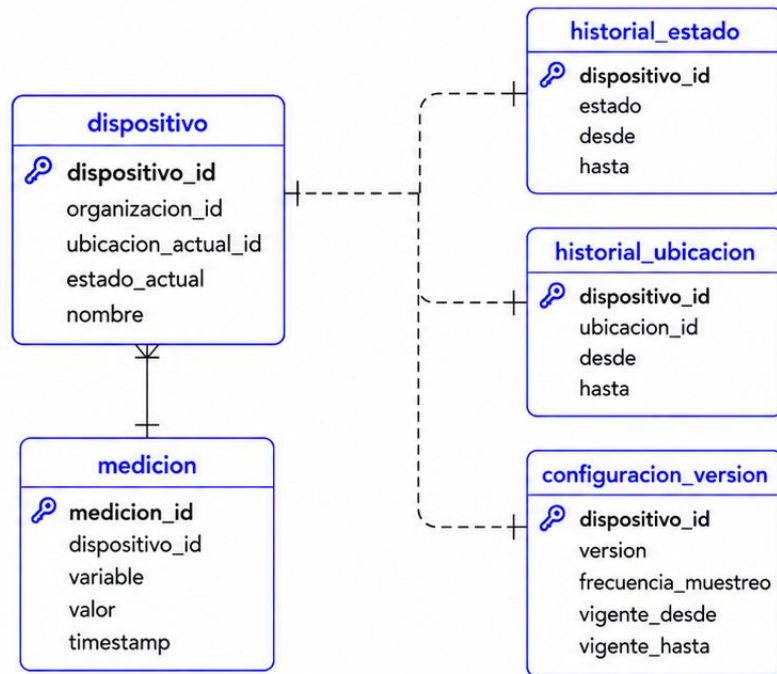
Estamos acercándonos a una estructura implementable, pero todavía estamos diseñando.

Paso 10

Revisar estados e historiales

Vamos a preguntarnos:

- ¿qué información cambia?
- ¿qué relaciones pueden cambiar?
- ¿qué datos requieren versiones?
- ¿qué períodos de vigencia necesitamos representar?



Cuando el pasado importa, el modelo debe poder representarlo.



Para ir cerrando..

Errores frecuentes

Problemas típicos al modelar

- Crear tablas antes de comprender el problema.
- Confundir entidades con atributos.
- No definir identificadores.
- Utilizar datos cambiantes como claves.
- Resolver incorrectamente relaciones N:M.
- No definir cardinalidades.
- Sobrescribir información cuando necesitamos conservar versiones.
- Incorporar entidades que el sistema realmente no necesita.



Crear tablas antes de comprender el problema.



Confundir entidades con atributos.



No definir identificadores.



Utilizar datos cambiantes como claves.



Resolver incorrectamente relaciones N:M.



No definir cardinalidades.



Sobrescribir información cuando necesitamos conservar versiones.



Incorporar entidades que el sistema realmente no necesita.



Un modelo puede parecer correcto hasta que intentamos utilizarlo.

Otro error frecuente

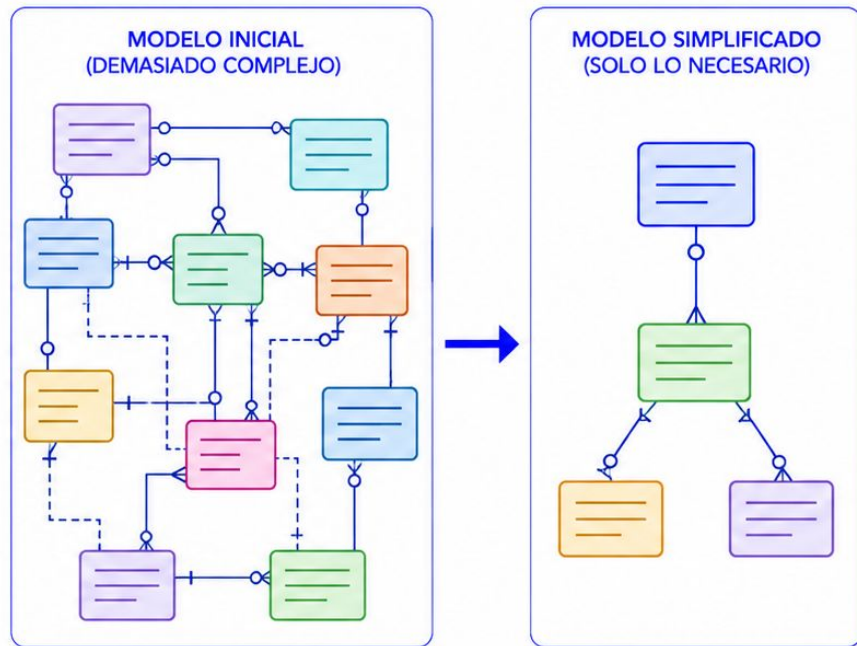
Intentar representar todo

Un sistema real contiene una enorme cantidad de conceptos.

No necesitamos modelarlos todos.

Debemos representar aquellos que son necesarios para:

- los requerimientos;
- las reglas;
- las consultas;
- las operaciones del sistema.

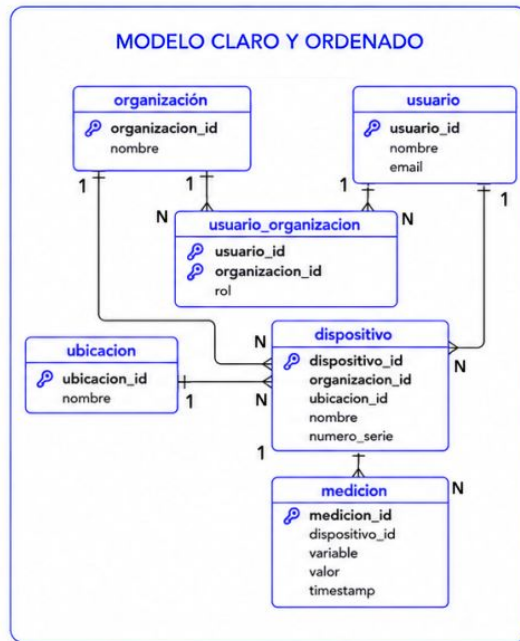


Un buen modelo no representa toda la realidad. Representa la parte de la realidad que necesitamos.

Buenas prácticas

Criterios simples para empezar bien

- Comprender primero el dominio.
- Utilizar nombres claros.
- Definir identificadores estables.
- Expresar relaciones mediante frases.
- Revisar cardinalidades.
- Representar reglas importantes.
- Considerar la evolución temporal cuando corresponda.
- Validar el modelo con ejemplos concretos.



CHECKLIST DE BUENAS PRÁCTICAS

- ✓ Nombres claros y consistentes.
- ✓ Identificadores únicos y estables.
- ✓ Relaciones con significado claro.
- ✓ Cardinalidades revisadas.
- ✓ Reglas y restricciones explícitas.
- ✓ Modelo simple y sin redundancias.
- ✓ Pensar en el cambio y el crecimiento.
- ✓ Validar con ejemplos reales.

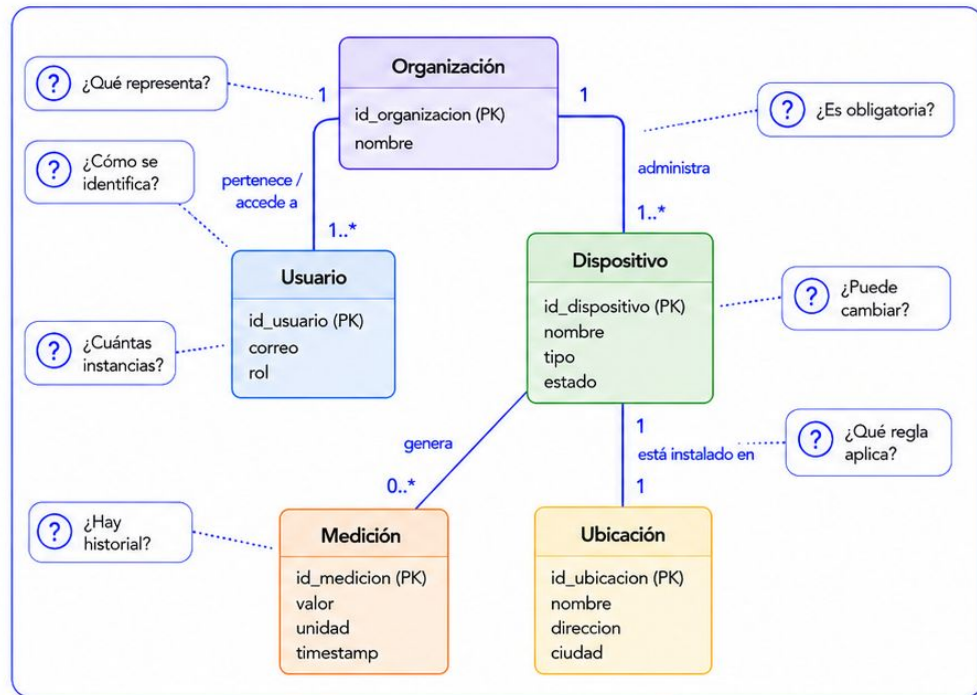


Un buen modelo debería poder explicarse.

Antes de implementarlo

Preguntas que podemos hacernos

- ¿Qué representa cada entidad?
- ¿Cómo identificamos cada registro?
- ¿Qué relaciones existen?
- ¿Cuántas instancias pueden relacionarse?
- ¿Qué relaciones son obligatorias?
- ¿Qué información puede cambiar?
- ¿Necesitamos representar versiones o períodos de vigencia?
- ¿Qué reglas deben cumplirse?



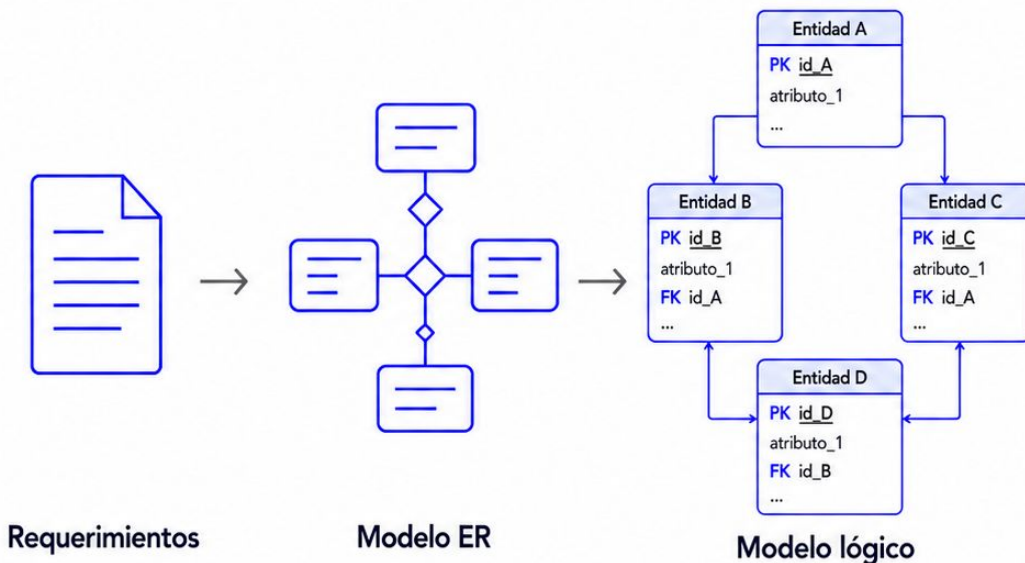
Modelar también significa hacer las preguntas correctas.

¿Qué observamos?

De los requerimientos a una estructura

Durante la clase vimos cómo:

- identificar conceptos del dominio;
- transformarlos en entidades;
- definir atributos;
- elegir identificadores;
- representar relaciones;
- establecer cardinalidades;
- incorporar restricciones;
- representar información que evoluciona;
- construir un modelo conceptual y lógico.

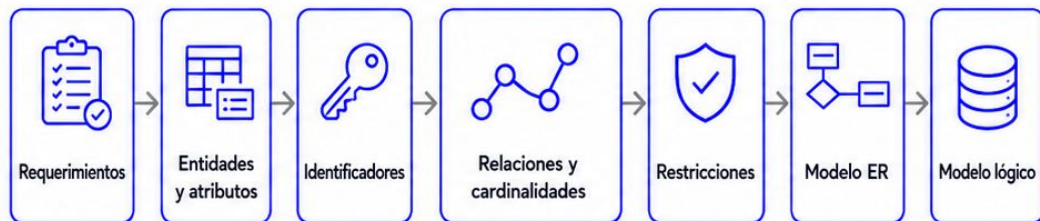


Antes de implementar una base de datos necesitamos decidir qué representa cada elemento y cómo se relaciona con los demás.

Resumen de la clase

Ideas principales

- Modelar datos implica representar los conceptos relevantes de un dominio.
- El modelado puede abordarse en niveles conceptual, lógico y físico.
- Las entidades representan elementos del problema y los atributos describen sus características.
- Los identificadores permiten distinguir cada instancia de manera inequívoca.
- Las relaciones y cardinalidades expresan cómo se vinculan las entidades.
- Las restricciones incorporan reglas que deben cumplirse.
- El modelo puede representar información que cambia mediante historiales y versiones.
- Los requerimientos son el punto de partida para construir y validar el modelo.



Un buen modelo transforma requerimientos del dominio en una estructura clara, consistente y preparada para ser implementada.

Próxima clase

Bases de Datos Relacionales y Lenguaje SQL

A partir del modelo diseñado hoy veremos cómo:

- representar los datos mediante tablas;
- utilizar filas, columnas y dominios;
- implementar claves primarias y foráneas;
- mantener integridad referencial;
- crear y modificar datos;
- realizar consultas SQL;
- combinar información de diferentes tablas.

Modelo conceptual (ER)



Tablas relacionales (PostgreSQL)

	organizacion
	id_organizacion (PK)
	nombre

	dispositivo
	id_dispositivo (PK)
	id_organizacion (FK)
	id_ubicacion (FK)
	nombre

	ubicacion
	id_ubicacion (PK)
	nombre

	medicion
	id_medicion (PK)
	id_dispositivo (FK)
	variable
	valor
	timestamp



Hoy diseñamos la estructura.

En la próxima clase comenzaremos a implementarla y consultarla.



Enlace a la encuesta



Muchas gracias ¿Dudas?